APRIL 2026 · ORANGE BOOK SERIES

Claude Code
Source Code Analysis

Dissecting Anthropic's AI Engineering Decisions from
510,000 Lines of Leaked Code

Inside Claude Code — Engineering Decisions Behind the World's Best AI Coding Tool

---

Based on: Claude Code v2.1.88 source code (leaked March 2026)

Audience: An architecture course for AI product builders

Prerequisite: Recommended to first read "Claude Code: From Beginner to Pro" Orange Book

Pages: ~80 pages · 12 chapters

Huashu

huasheng.ai

> Exclusive content for "AI Programming: From Beginner to Pro" Knowledge Planet members

This handbook is based on analysis of the TypeScript source code publicly exposed during the Claude Code v2.1.88 source code leak in March 2026. The analysis covers client-side tool code only and does not involve model weights, training data, or user information. The source code has been mirrored and archived in multiple GitHub repositories. AI tools iterate rapidly; the source code analysis reflects the state at the time of the leaked version.

This handbook is continuously updated. Get the latest version at: Feishu Document

Table of Contents

Claude Code: Source Code Analysis

PART I · The Big Picture

01 A Billion-Dollar Blueprint

02 Architecture Overview: The Skeleton of the Harness

PART II · Core Systems

03 System Prompt Engineering

04 The Tool System: 4 Primitives, 59 Tools

05 The Permission System: Trust by Design

06 The Memory System: Remember Preferences, Forget Code

07 Context Management: Surviving Long Conversations

08 Search: Why grep Beats RAG

PART III · Advanced Architecture

09 Multi-Agent Architecture: Run Like a Company

10 The Future in Feature Flags

11 Two Claude Codes: Internal vs External

PART IV · Methodology

12 The Harness Engineering Playbook

A Letter to Readers

Hi, I'm Huashu.

In March 2026, Claude Code's source code was accidentally leaked. 510,000 lines of TypeScript, fully exposed to the public eye.

For most people, this was just a tech news blurb. But for me, it was a rare learning opportunity. Claude Code isn't just a coding tool -- it's one of the most mature AI coding products available, and Anthropic's engineering team put serious thought into its architecture.

I spent two weeks going through the code line by line and wrote it all up in this book.

Let me be clear upfront: **this is not a source code reading journal**. Simply translating code into plain language wouldn't add much value. What I really care about are the design decisions behind every key junction. Why is the system prompt structured this way? Why did they choose this architecture for the tool system? How does the permission model balance security and usability? How does context management enable infinite conversations within a finite window?

The answers to these questions are valuable to anyone building AI products.

The real value of this book isn't about Claude Code itself. It demonstrates an AI application architecture that has been proven to work in a real product. When building your own AI product, you'll face many of the same problems: how to design tool invocations, how to manage context, how to handle permissions, how to coordinate multi-agent workflows. Anthropic's engineers have already walked these paths, and their choices and tradeoffs are worth studying carefully.

This book is the second in the "AI Programming Orange Book" series. The first, "Claude Code: From Beginner to Pro," teaches you how to use the tool. This one teaches you how it's built. If you've used Claude Code and want to understand what's happening underneath, or if you're building AI products and need architectural references, this book should help.

One caveat: the source code leak is a snapshot of a specific moment in time, and later versions will certainly have changed. The book analyzes architectural thinking and design principles -- things that are more durable than any specific line of code. If you find a detail that doesn't match the latest version, that's perfectly normal.

I also recommend the other three Orange Books: "Claude Code: From Beginner to Pro" is a hands-on getting-started manual, "Harness Engineering" covers methodology, and "Agent Skills" teaches you how to extend AI's capabilities. The four books are independent -- pick the ones that match your needs.

Finally, if reading this book sparks new ideas about AI product architecture, or if you end up using the design patterns mentioned here in your own projects, I'd love to hear about it.

Huashu
April 2026

01 A Billion-Dollar Blueprint

A Billion-Dollar Blueprint

On March 31, 2026, Anthropic accidentally shipped the complete Claude Code source code inside an npm release package. 1,902 TypeScript files, 510,000 lines of code, every design decision laid bare. This book is a deep reading of that blueprint.

What This Book Is About

If you've read my "Claude Code: From Beginner to Pro" Orange Book, that one teaches you how to drive the car. This one takes you under the hood.

But I won't be reading code line by line. That's a programmer's job, and besides, the source code gets updated constantly -- line-by-line analysis has a very short shelf life.

What I'm going to do is dissect **design decisions**. In building Claude Code, Anthropic made a huge number of non-obvious engineering choices. Why does search use grep instead of a vector database? Why does the memory system only store preferences, not code? Why does Auto mode run a second AI behind the scenes for safety reviews? Why Bun over Node.js?

Every choice has a reason behind it. Those reasons are more valuable than the code itself, because they're transferable. Regardless of what language or framework you use, these design principles can be directly applied.

Harness Engineering: An Emerging Discipline

A concept gaining serious traction in 2026: **harness engineering**.

What does it mean? How good an AI Agent is depends not just on how powerful the underlying model is, but on how well-built the system around the model is. This system is called the harness, and it includes tool invocation, permission control, memory management, context compression, safety guardrails, multi-agent coordination -- all the engineering components that turn AI from "capable but unpredictable" into "stable, reliable, and able to deliver."

Claude Code's source code is the most complete harness engineering textbook available to date.

There are plenty of AI coding tools on the market, all calling Claude or GPT APIs under the hood, yet their user experiences are worlds apart. The difference isn't in the model -- it's in the harness. The same Opus 4.6 model, wrapped in Cursor's harness, is one product; wrapped in Claude Code's

harness, it's another; wrapped in an amateur developer's wrapper, it's yet another. **The model determines the ceiling of capability; the harness determines how much of that ceiling gets realized.**

My assessment: Claude Code's excellence is 60% model capability, 40% harness engineering. This book is about that 40%.

Why This Book Matters

You might ask: the source code is public -- why not just read it directly?

Because 510,000 lines of code weren't designed to be read. There's no architecture document, no design specification, no README explaining why they did it this way instead of that way. The code tells you the what and the how, but not the **why**.

And the why is the most valuable part. Why doesn't search use a vector database? Because the LLM is smart enough. Why doesn't the memory system store code? Because code changes fast, and storing it actually does more harm than good. Why does the permission classifier use two stages instead of one? Because the first stage can greenlight most operations with just 64 tokens.

These whys aren't in the code. They're scattered across comment fragments, implied in design choices, hidden in PR number references and model version annotations. What this book does is collect those scattered whys and connect them into a coherent design philosophy.

Another reason is timeliness. AI tools iterate extremely fast; Claude Code's code changes daily. But design decisions evolve much more slowly than code. Anthropic isn't likely to swap grep for a vector database tomorrow, or remove the permission system's four-layer review. These architectural decisions will persist for a long time, and understanding them is far more valuable than understanding any single line of code.

The Leak

A brief recap. For the detailed version, see Appendix A in the v2.0 Orange Book.

On March 31, 2026, security researcher Chaofan Shou noticed that Claude Code's v2.1.88 package on npm was unusually large -- 59.8MB, several times bigger than normal. The reason: the build pipeline hadn't excluded `.map` files (source maps), causing the complete TypeScript source code to be bundled and published alongside the compiled output.

Scale of the leak: approximately 1,900 TypeScript files, 510,000 lines of code, 59 built-in tool definitions, 44 feature flags.

Anthropic urgently unpublished the version within hours, but multiple mirror repositories had already appeared on GitHub. The code spread across the internet.

The greatest irony: the source code contained a complete anti-leak system (Undercover Mode), yet the actual leak happened because a build script was missing a single line. And this was the second time -- the exact same accident had occurred in February 2025, when Claude Code first launched.

To be clear: **this leak did not involve model weights, training data, or user information**. What leaked was purely client-side tool code -- the CLI program running on your computer.

Aftermath of the Leak

The story didn't end there. Several noteworthy things happened within 48 hours:

Anthropic sent a DMCA notice to GitHub. Due to chain effects in the fork network, GitHub's enforcement **accidentally disabled over 8,100 repositories**, many of which were simply forks of Anthropic's own public repositories (containing skills and example code), completely unrelated to the leak. This blunder made bigger waves than the leak itself. Anthropic later narrowed the scope to 1 repository and 96 forks.

Claude Code's creator Boris Cherny publicly responded to the incident. His exact words are worth recording: "This was human error. There was a manual deploy step that should have been better automated. No one was fired -- this was an honest mistake. As a team, it's important to recognize that this is never an individual's fault -- it's a process, culture, or infrastructure issue."

"Blame the process, not the person." This statement says more about Anthropic's engineering culture than any technical analysis could.

Even more ironic was the possible root cause. The community discovered that Bun had a known bug (oven-sh/bun#28001, filed March 11) reporting that source maps were still being served in production mode despite documentation saying they should be disabled. The issue remains open. If this was indeed the cause, then Anthropic's toolchain (Bun) shipped a known bug that exposed their own product.

There was also a security incident: around the same time as the leak, malicious versions of axios (1.14.1 and 0.30.4) appeared on npm, containing a remote access trojan. Users who installed Claude Code during that specific time window may have pulled in the malicious dependency. This is a reminder: **supply chain security is not a theoretical threat**.

How to Read This Book

The book is divided into four parts:

Part I - The Big Picture (Chapters 1-2): Build an overall understanding. Learn Claude Code's tech stack, core architecture, and execution loop.

Part II - Core Systems (Chapters 3-8): Dissect the design decisions of six core systems, one by one. Each chapter follows the structure: what the system does → how it's implemented in source code →

why it was designed this way → what you can learn from it.

Part III - Advanced Architecture (Chapters 9-11): Multi-agent collaboration, unreleased features, internal vs external version differences. More cutting-edge content suited for readers with a deeper interest in AI product architecture.

Part IV - Methodology (Chapter 12): Distill insights from all previous chapters into reusable harness engineering principles.

You can read from start to finish or jump to chapters that interest you. Each chapter is relatively self-contained.

> **About Source Code References**: Code snippets and file paths cited in this book are based on the v2.1.88 version leaked in March 2026. Claude Code is continuously evolving, and actual code may have changed. But design decisions and engineering philosophies change much more slowly than code, which is why I focus more on the "why" than the "how."

By the Numbers

Before we begin, here are a few numbers to build intuition:

Metric	Count	Notes
TypeScript source files	~1,900	All in strict mode
Total lines of code	~510,000	Excluding dependencies
Built-in tools	59	From Read to TeamCreate
Feature Flags	44	Controlling unreleased features
Permission classifier	2 stages	64 → 4,096 token budget
Memory types	4	user/feedback/project/reference
Compression structure	9 sections	Structured information retention
Vector databases	0	Search runs entirely on grep

That last row is probably the most curiosity-inducing. 510,000 lines of code, zero vector databases. The logic behind this decision is explored in detail in Chapter 8.

02 Architecture Overview: The Skeleton of the Harness

Architecture Overview — The Skeleton of the Harness

Before diving into each subsystem, let's build an understanding of the overall architecture. Claude Code is not a simple API wrapper -- it's a carefully designed runtime system comprising multiple cooperating subsystems.

The Tech Stack: Every Choice Has a Reason

The first thing you see when opening the source code is the technology choices. Some are expected; some are quite surprising.

Component	Choice	Why
Runtime	**Bun**	Faster than Node.js, especially for cold starts and subprocess creation
UI Framework	**React + Ink**	Terminal UI state management needs are essentially the same as web frontend
Language	**Strict-mode TypeScript**	A 1,900-file project would collapse without a type system
Schema Validation	**Zod v4**	Runtime type checking to prevent AI from returning malformed data
Entry File	**main.tsx** (785KB)	Single-file bundle to reduce module resolution overhead
Heavy Module Loading	**Lazy loading**	OpenTelemetry, gRPC, etc. loaded on demand to avoid slowing startup

Why Bun Over Node.js

Claude Code frequently spawns subprocesses (every Bash tool call is a subprocess), performs heavy file I/O (Grep, Glob, Read tools), and handles concurrent requests (multiple agents working simultaneously). In these scenarios, Bun outperforms Node.js by a significant margin.

But the more critical factor is **cold start speed**. For a command-line tool, the delay between pressing Enter and seeing a response directly impacts the experience. Bun's cold start is 3-5x faster than Node.js. For a tool used hundreds of times a day, saving 200 milliseconds each time adds up noticeably.

Why Use React for a Terminal Interface

This is the most surprising choice. React for a terminal program?

But think about Claude Code's interaction complexity: real-time updating progress bars, collapsible code diffs, permission confirmation dialogs, multi-layered nested tool call displays, split-screen state display during multi-agent parallel work... The state management requirements for all of this are fundamentally the same problem as web frontend development. Using `console.log` to concatenate strings for all this would turn the code into unmaintainable spaghetti.

React's Ink framework lets Anthropic build the terminal interface with a component-based approach. Each UI element is a component; when state changes, it automatically re-renders. This is a **complexity management** choice, not technical showing off.

The community also discovered deeper performance optimizations after the leak. `ink/screen.ts` and `ink/optimizer.ts` borrow techniques from **game engines**: using Int32Array-backed character pools instead of string operations, encoding style metadata with bitmasks, and having a dedicated patch optimizer that merges cursor movements and cancels redundant hide/show operations.

There's also a self-clearing line-width cache that reduces `stringWidth` calculations by roughly 50x during token streaming. Claude Code may redraw dozens of times per second -- these optimizations are necessary for a smooth experience.

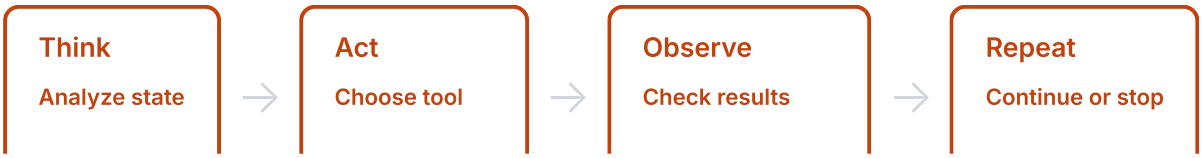
Why Zod for Schema Validation

TypeScript types only exist at compile time. At runtime, the JSON returned by AI could be any shape. Zod provides runtime type checking and transformation, ensuring data from the API matches the expected format.

This is especially important in AI applications. Model output is non-deterministic -- you can't assume it will return the correct format every time. Zod establishes a defense line between "non-deterministic AI output" and "deterministic program logic."

The Core Loop: TAOR

The heart of Claude Code is a loop called **TAOR**: Think → Act → Observe → Repeat.



Everything that happens after you type a message in the terminal is driven by this loop. The model first understands what you want (Think), then chooses a tool to execute an action (Act), observes the tool's returned results (Observe), and judges whether the task is complete. If not, it loops back to Think. A single task might take dozens of iterations to finish.

TAOR isn't a concept invented by Claude Code, but Claude Code's implementation has several interesting design details:

Tool calls are the only way to "act." The model can't directly manipulate the file system or run commands -- it must act indirectly through tools. This means all operations pass through a middleware layer where permission checking, logging, and security review can happen. This is the foundation for the permission system in Chapter 5.

Each loop iteration is a complete API call. Each Think-Act cycle requires sending the current context to the API, waiting for the model to return a tool call instruction, executing it, adding the result to the context, and making another API call. This explains why complex tasks consume so many tokens. It's not because the model is "overthinking" -- it's because the loop structure requires sending the full context at each step. This is also why context management (Chapter 7) is so critical.

The stopping condition is decided by the model itself. When does the loop stop? When the model judges the task is complete, it stops calling tools and outputs a text response instead. This is why giving Claude clear verification criteria is so important. If the requirement description is vague, the model doesn't know when "done" means done, and the loop keeps running.

The Five Subsystems

TAOR is the skeleton, but what makes Claude Code truly effective are the five subsystems built around it:

Subsystem	Responsibility	Code Volume	Chapter
System Prompt	Defines the AI's identity, capability boundaries, and behavioral guidelines	--	Ch. 3
Tool System	Registration, invocation, and permission management for 59 tools	~50,000 lines	Ch. 4
Query Engine	All LLM API calls, streaming, caching, and orchestration	~54,000 lines	--
Permission System	Security review and access control for operations	--	Ch. 5
Memory System	Persistent preferences and context across sessions	--	Ch. 6

Beyond these five core subsystems, there are several important auxiliary modules:

IDE Bridge: A bidirectional communication layer between VS Code/JetBrains and the CLI. Uses JWT for authentication and supports two-way message passing. The IDE can send tasks to the CLI, and the CLI can push status updates to the IDE. This is the foundation for Claude Code functioning as both a standalone CLI and an IDE plugin.

Context Manager: Responsible for structured compression when conversations get long, and for managing prompt caching strategies. Detailed in Chapter 7.

Multi-Agent Coordinator: Manages the creation, communication, isolation, and result merging of multiple agents. Detailed in Chapter 9.

Anatomy of a Request

Putting the above concepts together, here's what the complete journey looks like when you type "write me a sorting function" in Claude Code:

1. **System Prompt Assembly**

   The system loads static prompt segments from cache (identity definition, safety guidelines, tool descriptions), then appends dynamic segments (your CLAUDE.md, current directory info, memory files, git status). This might be a massive prompt of several thousand tokens.

2. **API Call (Think)**

   Your message, along with the system prompt and conversation history, is sent to the Claude API. The query engine handles streaming, and you see text appearing gradually.

3. **Tool Call (Act)**

   The model decides to first check the current directory's file structure (calling the Glob tool), then look for existing sorting-related code (calling the Grep tool). Each tool call passes through the permission system for review.

4. **Result Injection (Observe)**

   Tool return results are added to the conversation context. The model now knows the directory structure and existing code.

5. **Loop Continues (Repeat)**

   The model decides to use the Write tool to create a file, then the Bash tool to run tests. Each step is a new API call carrying all previous context.

6. **Completion**

   After tests pass, the model judges the task complete and outputs a summary. If the conversation triggers memory extraction conditions, a sub-agent is forked in the background to extract preference information worth remembering.

Looks simple? In reality, even a small task like "write a sorting function" might involve 3-5 TAOR iterations, each a complete API call. A complex task might have dozens of iterations consuming tens of thousands of tokens.

But that's the beauty of TAOR: **it's not executing a pre-written script -- it's making decisions in real time.** Each step adjusts strategy based on the latest observations. Sometimes it tries one approach, finds it doesn't work, and pivots to another path. That's not a bug; it's by design.

Code Scale in Perspective

Finally, some numbers to help you build intuition about this project's scale:

Metric	Count	Notes
Total source files	~1,900	All TypeScript
Total lines of code	~510,000	Excluding node_modules
Tool system	~50,000 lines	One of the most complex modules
Query engine	~54,000 lines	Largest single module (services/ directory)
Built-in tools	59	Each with independent permission control
Slash commands	~50	User-accessible shortcut commands
Feature Flags	44	Controlling unreleased features
src/ subdirectories	41	Highly modular project structure

This is a serious engineering project. Not a hobby project built by three to five engineers over a weekend, but a product-grade software system with sustained investment from a large team and careful design.

Someone on Hacker News commented that Anthropic's code looks "vibe coded" -- written with a "if it feels right, ship it" attitude. Whether that's fair is debatable, but it illustrates one point: **Claude Code's competitive advantage isn't in whether the code is elegant, but in whether the system design decisions are correct.** Getting a few key architectural choices right matters far more than writing every line of code beautifully.

Over the next ten chapters, we'll dissect these key decisions one by one.

03 System Prompt Engineering

System Prompt Engineering

When you type a message in Claude Code, the AI receives far more than just the few words you wrote. Your instruction is merely the tip of the iceberg -- beneath the surface lies a massive, meticulously assembled system prompt.

What the AI Actually Receives

The source code's `prompts.ts` file spans 911 lines, defining the complete system prompt construction process. The core function `getSystemPrompt()` returns an array of strings, each element a prompt segment. Concatenated together, they form everything the AI has read before you send your first message.

These instructions fall into several categories:

Category	Source Function	Content
Identity Definition	`getSimpleIntroSection()`	"You are Claude Code, Anthropic's official CLI tool"
Safety Guidelines	`CYBER_RISK_INSTRUCTION`	Security boundaries: allowed and prohibited operations
System Behavior	`getSimpleSystemSection()`	Markdown rendering, permission modes, hooks handling
Task Guidelines	`getSimpleDoingTasksSection()`	Don't over-engineer, don't fabricate data, don't delete files carelessly
Cautious Action	`getActionsSection()`	Confirm before high-risk operations, don't use destructive approaches
Tool Usage	`getUsingYourToolsSection()`	Prefer dedicated tools over Bash
Style Requirements	`getSimpleToneAndStyleSection()`	No emoji, concise and direct, proper code formatting
Output Efficiency	`getOutputEfficiencySection()`	Get to the point, act first then explain

The above are the **static** portions, shared across all users and cacheable across requests. After these come the **dynamic** portions: your CLAUDE.md configuration, current directory information, memory files, MCP server descriptions, git status, language preferences...

The static portion alone is estimated at 3,000-5,000 tokens. With the dynamic portions, a typical system prompt might be 5,000-10,000 tokens. Before you've said a single word, the AI has already read a small booklet.

The Dynamic Boundary: One Cut Creates the Caching Strategy

The source code contains a key constant:

```
export const SYSTEM_PROMPT_DYNAMIC_BOUNDARY =
  '__SYSTEM_PROMPT_DYNAMIC_BOUNDARY__'
```

This is a dividing line that cuts the entire system prompt into two halves. Above the line is static content, shared across all users and cacheable with `scope: 'global'` across organizations. Below the line is dynamic content, loaded independently for each user.

Why do this? **To save money.**

Claude API's prompt caching mechanism works like this: if the prompt prefix is identical between two requests, the second can reuse the first's cache without reprocessing. This means the static segments shared across all users only need to be processed once.

Source code comments explain the design intent quite directly:

```
/**
 * Session-variant guidance that would fragment the cacheScope:'global'
 * prefix if placed before SYSTEM_PROMPT_DYNAMIC_BOUNDARY. Each conditional
 * here is a runtime bit that would otherwise multiply the Blake2b prefix
 * hash variants (2^N). See PR #24490, #24171 for the same bug class.
 */
```

Translation: if any conditional content is placed above the boundary (e.g., displaying different text based on user type), cache hit rate drops exponentially. Each additional conditional branch doubles the number of cache variants (2^N). These two PRs (#24490 and #24171) fixed exactly this class of bug -- dynamic content accidentally placed in the static zone.

This is a very pragmatic engineering decision. **The physical layout of a prompt affects not just semantics but cache hit rates, directly impacting operational costs.**

Three Types of Prompt Sections

The source code defines three ways to register system prompt sections:

Type	Function	Caching Behavior	Use Case
Static Section	Direct string	Global cache	Identity definition, safety guidelines, task guidelines
Dynamic Section	`systemPromptSection()`	Cached by content hash	Environment info, memory, language preferences
Uncached Section	`DANGEROUS_uncachedSystemPromptSection()`	Not cached	MCP server instructions (can change at any time)

The name `DANGEROUS_uncached` isn't arbitrary. Source code comments explain why:

```
DANGEROUS_uncachedSystemPromptSection(
  'mcp_instructions',
  () => isMcpInstructionsDeltaEnabled()
    ? null
    : getMcpInstructionsSection(mcpClients),
  'MCP servers connect/disconnect between turns',
)
```

MCP servers might connect or disconnect mid-conversation, so their instructions can't be cached -- caching them might give the model stale information. But not caching means resending this portion with every API call, increasing costs. So Anthropic added the `DANGEROUS_` prefix as a warning: **think twice before using this function -- make sure caching truly isn't possible.**

Internal vs External Prompt Differences

The source code is full of `process.env.USER_TYPE === 'ant'` checks, creating significant differences between internal and external system prompts:

Code Style Instructions

The external version has no special code commenting requirements. The internal version adds four strict rules:

```
// Internal-only
- Default to writing no comments. Only add one when the WHY
  is non-obvious.
- Don't explain WHAT the code does, since well-named
  identifiers already do that.
- Don't reference the current task, fix, or callers.
- Before reporting a task complete, verify it actually works.
```

The last rule is particularly important. "Verify before reporting completion" -- the source code annotation marks this as "capy v8 thoroughness counterweight," meaning it's a countermeasure for the Capybara model (internal codename) v8's tendency to be "not thorough enough." **They found the model sometimes claims "done" without actually verifying, so they explicitly require verification in the prompt.**

Honesty Instructions

The internal version also has a lengthy anti-false-claims statement:

```
Report outcomes faithfully: if tests fail, say so with the
relevant output; if you did not run a verification step, say
that rather than implying it succeeded. Never claim "all tests
pass" when output shows failures...
```

This instruction also carries a model version annotation: "False-claims mitigation for Capybara v8 (29-30% FC rate vs v4's 16.7%)." Translation: Capybara v8's false claim rate is 29-30%, much higher than v4's 16.7%, so a stronger prompt is needed to suppress it.

This data is very revealing. It tells us: **even the most advanced models have non-trivial false claim rates.** Anthropic has a clear-eyed understanding of their model's flaws and uses prompt engineering to compensate.

Output Style

This is where the biggest differences lie. The external version is just 4 lines: "Get to the point, be concise, no fluff." The internal version is a full paragraph, with the core requirement being:

"You're writing text for a person to read, not logging to a console. Assume the user has been away and lost context -- they don't know what codenames or abbreviations you used along the way. Write so they can cold pick up."

There are also specific writing requirements: use flowing prose, avoid fragmented expressions, excessive em dashes, symbols and notation. Don't cram explanatory reasoning into table cells. Avoid semantic backtracking -- let the reader progress linearly without needing to re-parse earlier content.

Why doesn't the external version have these? Possibly because the external version needs to be more concise (most users are developers accustomed to code-style output), or possibly because it's still in A/B testing. The source code comments indeed note "un-gate once validated on external via A/B."

Five-Level Priority Override

The source code's `systemPrompt.ts` reveals a more complex design: the system prompt isn't a fixed template but a **five-level priority system**, overriding from highest to lowest:

Priority	Source	Use Case
1 (Highest)	**overrideSystemPrompt**	Test/debug mode, complete replacement
2	**Coordinator Mode**	Multi-agent coordination mode (detailed in Ch. 9)
3	**Agent System Prompt**	Domain-specific agent instructions
4	**Custom System Prompt**	User-specified via `--system-prompt`
5 (Lowest)	**Default System Prompt**	Standard Claude Code instructions

There's also a special `appendSystemPrompt` that gets appended to the end regardless of which level is active.

This design supports Claude Code's evolution from a single CLI into a multi-mode platform: regular users see the Default, sub-agents see the Agent prompt, Coordinator mode has its own completely different role definition, and developers can use override for complete replacement during testing.

Three Operating Modes

The source code actually implements three completely different system prompts, switching based on operating mode:

Mode A: Simplified Mode (CLAUDE_CODE_SIMPLE=1)

```
You are Claude Code, Anthropic's official CLI for Claude.
CWD: /path/to/project
Date: 2026-04-02
```

That's it. Three lines. Used for embedded scenarios (like being called as a subprocess), where full behavioral guidance isn't needed.

Mode B: Standard Interactive Mode

The full prompt detailed above -- 8 static sections + N dynamic sections, thousands of tokens.

Mode C: Proactive Mode (KAIROS/Assistant Mode)

The source code has a branching logic: if Proactive mode (KAIROS/Assistant mode) is detected, the entire system prompt switches to a completely different minimalist version:

```
if (proactiveModule?.isProactiveActive()) {
  return [
    `You are an autonomous agent. Use the available tools
     to do useful work.`,
    getSystemRemindersSection(),
    await loadMemoryPrompt(),
    envInfo,
    // ...
  ]
}
```

From "You are Claude Code, helping users with programming tasks" to "You are an autonomous agent, use tools to do useful work." The identity definition is completely different, and behavioral constraints are greatly simplified. This suggests KAIROS isn't an enhanced version of Claude Code but a conceptually different product -- an **AI that acts proactively without waiting for user instructions**.

Cache TTL Stability Engineering

An easily overlooked but very elegant design: cache TTL stability handling.

Claude API supports two cache TTLs: 5 minutes (default) and 1 hour (paid users). Whether a user qualifies for the 1-hour cache is **locked at session startup**:

```
let userEligible = getPromptCache1hEligible()
if (userEligible === null) {
  userEligible = process.env.USER_TYPE === 'ant' ||
    (isClaudeAISubscriber() && !currentLimits.isUsingOverage)
  setPromptCache1hEligible(userEligible)  // Lock it in
}
```

Why lock it? Because if a user transitions from free to paid (or vice versa) mid-session, the TTL would change from 5 minutes to 1 hour. A TTL change would alter the cache key, invalidating the entire prompt cache and wasting approximately 20,000 tokens in reprocessing.

So Anthropic chose to determine the TTL at session start and keep it unchanged throughout the session. **Stability matters more than precision.** The cost of occasionally using the wrong TTL is far less than the cost of mid-session cache invalidation.

Lessons for AI Product Builders

> **KEY TAKEAWAY**
>
> **A system prompt is an engineering system, not a piece of text.** It needs version management, effect testing, and performance optimization, just like code. Claude Code's prompt has an explicit caching strategy, modular structure, A/B testing annotations, and model version compatibility handling. This isn't a matter of "writing a good instruction" -- it's "building a maintainable instruction system."

Several key takeaways:

Caching determines prompt structure. Don't finalize content and then arrange it randomly. First draw the cache boundary -- put immutable content up front, variable content at the end. Every misplaced conditional branch wastes cache budget.

Use prompts to compensate for model weaknesses. Anthropic doesn't pretend their model is perfect. Capybara v8 has a 30% false claim rate? Add anti-false-claims instructions to the prompt. Model isn't thorough enough? Add "must verify before reporting completion." **Know your model's weaknesses, then use prompt targeting to patch them.**

The internal version reveals the ideal state. Those strict requirements in the internal version represent what Anthropic believes AI should ideally do. If you're designing your own AI product, the internal version's prompt is a great reference benchmark.

Different modes need different prompts. Proactive mode's prompt is completely different from normal mode. Don't try to cover all use cases with one universal prompt. Different scenarios call for different identity definitions and behavioral constraints.

The caching system is more fragile than you think. The community discovered a real case: a prompt cache regression bug appeared around Claude Code v2.1.76. Two technical causes compounded: changing attestation data in each request caused cache hash invalidation, and anti-distillation injected fake tool definitions that varied between requests.

Normally, Turn 4 costs should drop to 13% of Turn 1 (cache hit), but in the buggy version, costs actually inflated from $0.04 to $0.40. **Any tiny change in the cache prefix invalidates the entire cache**, causing 10-20x cost inflation. This isn't a Claude Code-specific issue -- it's a trap all AI products using prompt caching should be wary of.

04 The Tool System: 4 Primitives, 59 Tools

The Tool System — Four Primitives, Fifty-Nine Tools

Claude Code's tool system is probably the largest part of the entire source code -- roughly 50,000 lines of code across 59 independent tool directories. But all these tools ultimately do just four things.

Four Capability Primitives

59 tools sounds like a lot, but if you step back, all capabilities reduce to 4 primitives:

Primitive	What It Does	Representative Tools
Read	Retrieve information	FileRead, Grep, Glob, WebFetch, WebSearch
Write	Create or modify	FileWrite, FileEdit, NotebookEdit
Execute	Run commands	Bash, PowerShell, REPL
Connect	Interface with external systems	MCP tools, Agent, SendMessage, TeamCreate

This categorization isn't my invention -- it's distilled from the source code's design intent. Tools are divided along these four dimensions for permission levels: Read is typically low-risk (doesn't change anything), Write and Execute are medium-to-high risk (changing files or system state), and Connect involves external interactions.

The Complete Tool Directory

The source code's `src/tools/` directory contains 59 tool directories. Categorized by function:

Category	Tool Count	Representative Tools
File Operations	5	FileRead, FileWrite, FileEdit, Glob, Grep
Command Execution	3	Bash, PowerShell, REPL
Web Access	3	WebFetch, WebSearch, WebBrowser
Agent Coordination	9	Agent, SendMessage, TeamCreate, TeamDelete, TaskCreate/Get/List/Update/Stop
Planning & Worktree	4	EnterPlanMode, ExitPlanMode, EnterWorktree, ExitWorktree
MCP-related	4	MCPTool, McpAuth, ListMcpResources, ReadMcpResource
Skills & Config	4	Skill, DiscoverSkills, Config, ToolSearch
Internal-only	7+	Tungsten, Monitor, ReviewArtifact, SuggestBackgroundPR...
KAIROS-only	3	PushNotification, SendUserFile, SubscribePR
Other	Several	Sleep, Brief, TodoWrite, NotebookEdit, RemoteTrigger...

Bash: The Universal Adapter

Among the 59 tools, Bash is the most special. It's a **universal adapter** that lets Claude use every command-line tool available to human developers.

No need for dedicated integrations for each programming language. No need for framework-specific plugins. `npm install`, `python test.py`, `git push`, `docker build` -- Bash handles everything. This is also why Claude Code works in projects of virtually any tech stack, unlike some IDE plugins that only support specific languages.

But the source code places strict limits on Bash usage. The system prompt explicitly requires:

```
Do NOT use the Bash to run commands when a relevant
dedicated tool is provided.
- To read files use Read instead of cat, head, tail
- To edit files use Edit instead of sed or awk
- To search for files use Glob instead of find or ls
- To search content use Grep instead of grep or rg
```

Why not let the AI use Bash for everything? Because **dedicated tools are more controllable than universal ones**. The Read tool can only read files, so the permission system's review is straightforward. Bash commands can do anything, so the permission system needs to understand command semantics to judge safety -- a much harder problem.

So Bash is the last resort, not the first choice. Wherever a dedicated tool is available, always use the dedicated tool.

Bash's Security Analysis Engine

The Bash tool's security analysis is very granular. The source code includes a complete command semantics analysis module (`commandSemantics.ts`) that understands the nature of each command in a pipeline:

```
// Search: find, grep, rg, ag, ack, locate, which
// Read: cat, head, tail, less, jq, awk, cut, sort
// List: ls, tree, du
// Semantically neutral: echo, printf, true, false
```

Key design: **every part of a pipeline must be a search/read command** for the entire pipeline to be considered read-only. For example, `cat file | jq .field` is safe (both are reads), but `find . | xargs rm` is not (contains a write operation).

There's also AST-level security parsing (`bashSecurity.ts`) that doesn't just look at command names but parses the entire command's syntax tree, detecting output redirections, command substitutions, and other patterns that might hide dangerous operations.

Deferred Tools: The Wisdom of Lazy Loading

This is a clever optimization. The source code includes a `ToolSearchTool` whose purpose is to let the model load tool definitions on demand.

Why is lazy loading needed? Because each tool's description (name, parameter schema, usage instructions) consumes system prompt tokens. If all 59 tools were fully expanded, they could take up tens of thousands of tokens. For most tasks, users only need 5-10 of them.

The Deferred Tools mechanism works like this: the system initially only tells the model "these tools are available" (a name list) without providing complete parameter definitions. When the model determines it needs a particular tool, it first calls `ToolSearch` to get the full definition, then calls the actual tool.

This design is especially important in MCP scenarios. If you connect 5 MCP servers, each exposing 10 tools, that's 50 additional tool definitions. Loading them all could consume 20,000-30,000 tokens of context. Deferred loading means most tools occupy zero context space when not needed.

Source code comments mention that each MCP server's tool definitions consume approximately 4,000-6,000 tokens. **More tools isn't always better -- each one has a cognitive cost.**

Plugin Architecture

Each tool is an independent directory containing the following files:

```
src/tools/BashTool/
├── BashTool.tsx      # Tool implementation (with UI component)
├── prompt.ts         # Tool description and parameter definitions
├── toolName.ts       # Tool name constant
├── bashSecurity.ts   # AST-level security parsing
├── commandSemantics.ts  # Command semantics analysis
└── utils.ts          # Utility functions
```

This structure means adding a new tool is very simple: create a new directory, implement the standard interface, register it in the tool list. No core code modifications needed. This is why Claude Code can add new tools at a rapid pace, growing from an initial dozen to the current 59.

Every tool implements a unified `Tool` interface, including:

- **name**: Tool name
- **description**: Tool description for the model
- **parameters**: Parameter format defined by Zod schema
- **execute**: Actual execution logic
- **permissionContext**: Context needed for permission checking

The Elegance of FileEdit

Many people's first instinct is: to have AI edit files, just use `sed` or rewrite the whole file. Claude Code's designers clearly thought otherwise.

FileEdit's core mechanism is **unique matching**. When you call the Edit tool, you provide an `old_string` (content to replace) and `new_string` (replacement content). The system searches the file for `old_string`; if it finds more than one match, it returns an error:

```
if (matches > 1 && !replace_all) {
  return {
    result: false,
    message: `Found ${matches} matches of the string
to replace, but replace_all is false. To replace
all occurrences, set replace_all to true. To replace
only one occurrence, please provide more context to
uniquely identify the instance.`,
    errorCode: 9,
  }
}
```

This design solves the most common accident when AI edits files: **editing the wrong location**. When a function name appears 5 times in a file and the AI only wants to change one occurrence, the unique matching constraint forces the AI to provide more context (like including surrounding lines of code) until it can precisely locate the target. This is much more reliable than sed's line-number matching. Line numbers change during file editing, but text content context doesn't.

What if you genuinely want batch replacement? Set `replace_all` to `true`. The typical scenario for this parameter is renaming variables -- a variable name appears 20 times in a file and you really do want to replace all of them. The prompt explicitly tells the model: "Use `replace_all` for replacing and renaming strings across the file."

There's also an easily overlooked safeguard: **you must Read before you can Edit**. The source code checks `readFileState`; if the file hasn't been read yet, editing is refused outright. This prevents the AI from blindly modifying files without knowing their contents, and ensures the unique matching check is based on the file's latest state.

```
const readTimestamp =
  toolUseContext.readFileState.get(fullFilePath)
if (!readTimestamp || readTimestamp.isPartialView) {
  return {
    result: false,
    message: 'File has not been read yet.
      Read it first before writing to it.',
    errorCode: 6,
  }
}
```

Going further, the source code also checks whether the file was modified after it was read (via timestamp comparison). If the user or a linter modified the file after the AI read it, Edit requires the AI to re-read, preventing replacements based on stale content.

Why is this better than sed? `sed` is a line-oriented text processing tool using regular expressions. AI-generated regex frequently breaks -- escape characters, greedy matching, multiline mode, each is a

pitfall. The Edit tool uses exact string matching, no regex needed. The AI just needs to copy-paste the text it just read and write the modified version. This takes the error rate from "frequently problematic" down to "almost never a problem."

Tool Result Size Control

This is an easily overlooked but critical mechanism. Each tool has a `maxResultSizeChars` property defining the maximum character count for tool return results:

```
/**
 * Maximum size in characters for tool result
 * before it gets persisted to disk.
 * When exceeded, the result is saved to a file
 * and Claude receives a preview with the file
 * path instead of the full content.
 *
 * Set to Infinity for tools whose output must
 * never be persisted (e.g. Read, where persisting
 * creates a circular Read→file→Read loop and the
 * tool already self-bounds via its own limits).
 */
maxResultSizeChars: number
```

Most tools have a threshold of 100,000 characters. When a tool's return result exceeds this threshold, the system doesn't simply truncate it -- it saves the complete result to a temporary file on disk, then puts only a summary and file path in the conversation. Claude can then read the full result via the Read tool when needed.

Why not just truncate? Because truncation might lose critical information. The save-to-disk approach preserves complete data while keeping it out of the context window. This is an approach of **treating the context window as a scarce resource to be managed**.

But there's one exception: FileReadTool's `maxResultSizeChars` is set to `Infinity`. Source code comments explain why: if the Read tool's result were saved to a file, Claude would need to use the Read tool to read that file, which might produce another large result that gets saved to a file... creating an infinite loop. The Read tool controls output size through its own line limit (default 2,000 lines), without needing the external persistence fallback.

This design illustrates a point: **the context window is one of the most precious resources in an AI product**. If a single Grep search returns 100,000 characters that fill up the context, subsequent conversation quality will inevitably decline. The size control mechanism separates tool results' "storage" from their "visibility." Data isn't lost, but it doesn't occupy precious context space.

ToolSearch Matching Algorithm

Earlier we introduced the Deferred Tools concept. Here we'll look deeper at ToolSearch's matching logic. When the model calls ToolSearch, there are two query modes:

Exact selection mode: When the query starts with `select:`, it matches directly by name. You can select multiple tools with commas, e.g., `select:Read,Edit,Grep`. This is the fast path when the model already knows what tool to use.

Keyword search mode: When not starting with `select:`, it enters keyword search. ToolSearch needs to understand tool name semantics. The source code has two key name resolution strategies:

For regular tool names (CamelCase format), split by camelCase boundaries:

```
// Regular tool - split by CamelCase and underscores
const parts = name
  .replace(/([a-z])([A-Z])/g, '$1 $2')
  .replace(/_/g, ' ')
  .toLowerCase()
  .split(/\s+/)
  .filter(Boolean)
// FileEditTool → ['file', 'edit', 'tool']
```

For MCP tool names (`mcp__server__action` format), split by double and single underscores:

```
if (name.startsWith('mcp__')) {
  const withoutPrefix =
    name.replace(/^mcp__/, '').toLowerCase()
  const parts =
    withoutPrefix.split('__').flatMap(p => p.split('_'))
  // mcp__slack__send_message
  //   → ['slack', 'send', 'message']
}
```

Search also supports a `+` prefix to mark required keywords (e.g., `+slack send` requires the tool name to contain "slack"), and batch matching by MCP prefix (e.g., querying `mcp__slack` returns all Slack-related tools).

Search scoring considers tool name token matches, keyword matches in tool descriptions, and an optional `searchHint` field. This lets the model describe needs in natural language (e.g., querying "modify file"), and ToolSearch returns the FileEdit tool.

Lessons for AI Product Builders

> **KEY TAKEAWAY**
>
> **The best way to control AI isn't writing longer prompts -- it's designing better tools.** Tool definitions determine what AI can and can't do. Each tool's parameter schema is a set of constraints. AI can only operate within those constraints.

The universal tool is a fallback, not the first choice. Bash can do everything, but precisely because of that, it's the hardest to control. Prioritize tools with clear functionality and well-defined permissions. The universal tool is only for scenarios that dedicated tools don't cover.

Tools have token costs. Each tool description occupies context space. Loading all 59 tools is expensive. Deferred loading is a necessary optimization, especially in MCP scenarios where tool counts scale up.

Plugin architecture supports rapid iteration. Adding new tools doesn't require touching core code, letting teams develop different tools in parallel. For a fast-iterating AI product, this architectural flexibility is crucial.

05 The Permission System: Trust by Design

The Permission System — Trust by Design

When you use Auto mode, you might think every operation gets a free pass. In reality, two AIs are running behind the scenes: one executing your task, and another dedicated to reviewing whether the first one's operations are safe.

Three Permission Modes

Claude Code has three permission modes, clearly defined in the source code:

Mode	Behavior	Use Case
default / interactive	Prompts for confirmation on every dangerous operation	Daily use, safety-first
auto	ML classifier automatically judges safety	Experienced users, efficiency-first
yolo / bypassPermissions	Skips all confirmations	Non-interactive scripts, complete trust

Most people care about Auto mode. How does it decide whether an operation is safe?

The Four-Layer Review Pipeline

Auto mode doesn't simply let the AI decide for itself. It's a **four-layer pipeline**, each layer like a security checkpoint, progressively stricter:

1. **Rule Matching**

   Checks user-configured permission rules. If an operation has been explicitly allowed or denied (e.g., "allow all git commands" in settings.json), it follows the rule directly without proceeding to later steps.

2. **Low-Risk Skip**

   Certain operations are inherently low-risk (like reading files or searching code) and pass through directly. No need to waste an AI classifier call to determine "is reading a file safe."

3. **Allowlist Pass-Through**

   Read-only tools (Read, Grep, Glob, etc.) are on the allowlist and pass through directly. These tools, by design, never modify anything.

4. **ML Classifier Judgment**

   Only operations not caught by the first three layers reach the independent Claude Sonnet safety classifier. This classifier has its own dedicated system prompt, completely different from the main AI, specifically designed to assess operation risk levels.

Why this design? **Performance and cost.** Each ML classifier call is an additional API call with latency and expense. The four-layer pipeline ensures only operations that genuinely need judgment reach the fourth layer. The vast majority of everyday operations are handled in the first three layers.

The YOLO Classifier

The fourth layer's ML classifier is called `yoloClassifier` in the source code. Its implementation is in `src/utils/permissions/yoloClassifier.ts`.

The classifier has its own system prompt, stored in a separate txt file (`auto_mode_system_prompt.txt`). It receives information about the operation the main AI wants to execute, then outputs one of three judgments:

Judgment	Meaning	Subsequent Action
Allow	Operation is safe, proceed	Execute directly
Soft Deny	Risky, needs confirmation	Prompt user for confirmation
Hard Deny	Dangerous operation, block	Refuse to execute

The classifier actually employs a **two-stage judgment**, one of the most elegant designs in the source code:

Stage 1: Quick Judgment (64-token budget)

The classifier first makes a preliminary judgment with a minimal token budget (only 64 tokens). The output format is simple: `<block>yes/no</block>`. If Stage 1 judges "safe," it passes through directly without entering Stage 2.

Why only 64 tokens? Because most operations are obviously safe (reading files, searching code) and don't need deep reasoning. 64 tokens is enough for a yes/no output.

Stage 2: Deep Reasoning (4,096-token budget)

Only operations that Stage 1 flags as "possibly dangerous" enter Stage 2. This time, 4,096 tokens are allocated, requiring the classifier to output detailed reasoning and rejection rationale: `<block>yes/no</block><reason>explanation</reason>`.

The benefit of two stages is **cost optimization**: safe operations are greenlit at Stage 1 (consuming minimal tokens), while only suspicious operations need Stage 2's deep analysis. According to source code statistics, most operations don't need to enter Stage 2.

Several other technical details are also critical:

Temperature set to 0. Both stages use temperature 0, ensuring identical inputs produce identical outputs. Security judgments don't need creativity; they need determinism.

CLAUDE.md is injected into the classifier context. The classifier doesn't just look at the operation itself -- it also sees your CLAUDE.md configuration. If you wrote "allow any operations on /tmp" in your CLAUDE.md, the classifier considers this as user intent. This means CLAUDE.md influences not only the main AI's behavior but also security judgments.

Internal and external versions use different permission templates. The source code loads two permission template sets: `permissions_external.txt` and `permissions_anthropic.txt`. External users' custom rules **replace** default rules; internal employees' custom rules **append** to enterprise default rules.

Transcript format compression. The conversation history the classifier sees isn't the complete raw messages but a compact format. Each tool call can customize its own classifier input format (e.g., FileEdit sends only the file path but omits file contents), reducing token consumption.

The Circuit Breaker

The source code includes an elegant `denialTracking.ts` module implementing a circuit breaker:

```
export const DENIAL_LIMITS = {
  maxConsecutive: 3,
  maxTotal: 20,
} as const

export function shouldFallbackToPrompting(
  state: DenialTrackingState
): boolean {
  return (
    state.consecutiveDenials ≥ DENIAL_LIMITS.maxConsecutive ||
    state.totalDenials ≥ DENIAL_LIMITS.maxTotal
  )
}
```

The rule is simple: **3 consecutive denials, or 20 total denials, and Auto mode automatically degrades to manual confirmation mode.**

Why is a circuit breaker needed? Imagine a scenario: the main AI is executing a legitimate task, but the classifier repeatedly misjudges it as dangerous. Without circuit breaking, the AI would endlessly retry and get denied, stuck in a dead loop. The circuit breaker lets the system degrade gracefully when things go wrong, rather than getting stuck.

A successful operation resets the consecutive denial counter (but not the total counter). This means occasional denials won't trigger the breaker, but frequent denials definitely will.

Protected Files and Path Defense

The source code defines a series of protected files that no operation is allowed to modify:

- `.gitconfig` -- Git global configuration
- `.bashrc` / `.zshrc` -- Shell configuration
- `.claude.json` -- Claude Code's own configuration
- `.mcp.json` -- MCP server configuration

These files are protected because modifying them could cause persistent, cross-session harm. For example, if AI modified `.bashrc` to add a malicious command, that command would execute every time the user opens a terminal.

Path traversal defense is also comprehensive. The source code checks for unicode encoding, case transformations, backslash injection, and other common path traversal attack techniques. This prevents the AI (or attackers working through prompt injection) from using paths like `../../../.bashrc` to circumvent protections.

Permission Explanation Text

An easily overlooked detail: when the classifier denies an operation, it doesn't just say "not allowed." It also generates a human-readable explanation telling the user why the operation was deemed dangerous.

This explanation text is generated by another independent LLM call. Anthropic believes that if you're going to deny a user's operation, you should at least tell them why, and the reason should be natural language a person can understand, not a technical error code.

Dangerous Permission Stripping

This addresses a subtle security issue: users may have configured broad permission rules during daily use, such as `Bash(*)` (allow all bash commands) or `Bash(python:*)` (allow all python commands). In

manual confirmation mode, these rules are fine -- users see the operation content and confirm each time. But in Auto mode, these rules **bypass the classifier**.

Why? Because in Auto mode's four-layer pipeline, the first layer is rule matching. If the user's rule says "allow all Bash commands," then any Bash operation gets greenlit at layer one and never reaches the fourth layer's ML classifier. The classifier's security review becomes effectively useless.

The source code's solution is elegant: upon entering Auto mode, it automatically **strips** these dangerously broad rules and stashes them away; upon exiting Auto mode, it **restores** them.

```
export function stripDangerousPermissionsForAutoMode(
  context: ToolPermissionContext,
): ToolPermissionContext {
  // Collect all alwaysAllowRules
  const rules: PermissionRule[] = []
  for (const [source, ruleStrings] of
    Object.entries(context.alwaysAllowRules)) {
    // ...parse each rule
  }
  // Find dangerous rules among them
  const dangerousPermissions =
    findDangerousClassifierPermissions(rules, [])
  if (dangerousPermissions.length === 0) {
    return context // No dangerous rules, return as-is
  }
  // Strip dangerous rules, stash in strippedDangerousRules
  return {
    ...removeDangerousPermissions(
      context, dangerousPermissions),
    strippedDangerousRules: stripped,
  }
}
```

Which rules are considered "dangerous"? The source code defines detailed matching logic. For Bash, the following patterns are all stripped:

- `Bash` (no arguments, i.e., allow all commands)
- `Bash(*)` (wildcard, same as above)
- `Bash(python:*)` (allow all python commands -- python can execute arbitrary code)
- `Bash(node:*)`, `Bash(ruby:*)`, and other broad script interpreter rules

Upon exiting Auto mode, the `restoreDangerousPermissions` function restores the stashed rules. The user's configuration isn't permanently modified -- it's just temporarily shielded during Auto mode.

The elegance of this design lies in: **it doesn't trust that user permission configurations remain safe in an automated environment**. Broad rules set during manual usage could become security vulnerabilities

in unattended Auto mode. The system automatically adapts security levels during mode switches rather than leaving that responsibility to the user.

The Permission Explainer

When the classifier determines an operation needs user confirmation, users see more than just an "allow/deny" button. The system simultaneously displays a human-readable explanation to help users make informed decisions. This explanation is generated by an independent LLM call.

This module is called `permissionExplainer` in the source code. Its implementation is interesting. First, the tool definition forces the model to output in structured format using `tool_choice`, ensuring parseable results every time:

```
const EXPLAIN_COMMAND_TOOL = {
  name: 'explain_command',
  input_schema: {
    properties: {
      explanation: {
        description:
          'What this command does (1-2 sentences)',
      },
      reasoning: {
        description:
          'Why YOU are running this command.
           Start with "I"',
      },
      risk: {
        description:
          'What could go wrong, under 15 words',
      },
      riskLevel: {
        enum: ['LOW', 'MEDIUM', 'HIGH'],
        description:
          'LOW (safe dev workflows),
           MEDIUM (recoverable changes),
           HIGH (dangerous/irreversible)',
      },
    },
  },
}
```

Note the `reasoning` field design: it requires starting with "I," explaining from the AI's own perspective why it wants to execute this operation. This isn't an arbitrary formatting requirement -- it forces the model to generate first-person explanations ("I need to check the test results") rather than dry technical descriptions. Users find it more natural to read and easier to judge whether the operation matches expectations.

The `risk` field is limited to 15 words. This forces the model to distill the risk to its core, rather than writing lengthy dissertations listing every possible risk. Users making permission decisions need quick judgments, not papers.

The three-level risk definition is also practical:

Level	Meaning	Typical Operations
LOW	Safe development workflows	Running tests, formatting code
MEDIUM	Recoverable changes	Installing packages, modifying config files
HIGH	Dangerous/irreversible operations	Deleting files, force push

The system also injects recent conversation context into the evaluation request (last 3 AI messages, up to 1,000 characters). This lets the risk evaluator understand not just the operation itself but its context. The same `rm` command might be LOW risk in the context of "cleaning up temp files" but could be rated HIGH without any context.

Users can disable this feature via configuration (`permissionExplainerEnabled`), but it's enabled by default. Anthropic's design philosophy: **if you're going to deny or question a user's operation, at least provide a reason they can understand**.

Zig-Layer DRM: Compile-Level Security

The community discovered a surprising security layer after the leak: Claude Code's API requests contain a `cch=00000` placeholder. Before the request leaves the process, **Zig compiled code** in Bun's internals replaces these five zeros with a computed cryptographic hash. The server validates this hash to confirm the request comes from an authentic Claude Code binary.

Why Zig instead of JavaScript? JavaScript can be monkey-patched at runtime -- you can override any function, intercept any HTTP request. But Zig code compiled into Bun can't be modified at runtime without recompiling the entire runtime. This is essentially **DRM for API calls**, implemented at the HTTP transport layer.

23 Bash Security Checks

`bashSecurity.ts` contains 23 numbered security checks covering many non-obvious attack surfaces: Zsh equals-sign expansion defense (`=curl` executes curl in Zsh), Unicode zero-width space injection, IFS null-byte injection, 18 blocked Zsh builtins, and malformed token bypasses found during HackerOne audits. Each one has a real attack scenario behind it.

Lessons for AI Product Builders

> **KEY TAKEAWAY**
>
> **Security system investment should match core feature investment.** Claude Code's permission system spans multiple files, thousands of lines of code, an independent ML classifier, Zig-layer DRM, 23 Bash security checks, circuit breakers, and path defense. This isn't a feature built over a weekend -- it's a core system with continuous investment.

Multi-layer defense, progressive filtering. Don't put all judgment pressure on a single layer. Use simple rules for simple judgments (allowlists, rule matching) and complex methods for complex judgments (ML classifier). This approach is both efficient and reliable.

Security and efficiency are not opposites. The four-layer pipeline design means most operations are greenlit in the first three layers, and users barely notice the security checks. Only truly suspicious operations trigger the fourth layer's AI judgment, and waiting two extra seconds at that point is completely acceptable. A good security system is invisible.

Design degradation paths. The circuit breaker's existence shows Anthropic acknowledges the classifier isn't perfect. Rather than pursuing a perfect classifier (impossible), design good fallback behavior for when things go wrong. Consecutive denials trigger a switch to manual confirmation -- simple, reliable, and the user doesn't get stuck.

Protect persistent files. AI modifying temporary files has limited impact. But modifying shell configuration, system configuration -- these persistent files have cross-session, even cross-project effects. Protection levels for these files should be far higher than for ordinary files.

Permission policies must adapt to mode switches. Broad permissions set by users in manual mode could become security vulnerabilities in auto mode. A good system automatically adapts security levels during mode transitions rather than leaving that judgment to users. Dangerous permission stripping embodies this thinking.

Give users information for decisions, not just buttons. An "allow/deny" dialog that doesn't tell users what the operation does, why it needs to run, or what could go wrong leads to either blind allowing or blind denying. The risk evaluator's structured explanation turns permission confirmation from an "interruption" into an "informed decision."

06 The Memory System: Remember Preferences, Forget Code

The Memory System — Remember Preferences, Forget Code

Claude Code's auto memory might be one of the most impressive features you notice after using it. But looking at the source code reveals that the tradeoffs between "what to remember" and "what to forget" are far more nuanced than they appear.

Physical Implementation

The memory system's implementation is remarkably simple. No database, no vector store, no complex indexing. Just a directory on the file system:

```
~/.claude/projects/<project-slug>/memory/
├── MEMORY.md          # Index file, auto-loaded each session
├── user_role.md       # User identity info
├── feedback_testing.md # Behavioral feedback
├── project_status.md  # Project status
└── reference_docs.md  # External resource pointers
```

The core entry file is `MEMORY.md`, with hard constraints defined in the source code:

```
export const ENTRYPOINT_NAME = 'MEMORY.md'
export const MAX_ENTRYPOINT_LINES = 200
export const MAX_ENTRYPOINT_BYTES = 25_000
```

200 lines, 25KB. These are MEMORY.md's physical limits. Exceeding either triggers truncation with an appended warning message.

Why so strict? Because MEMORY.md is fully injected into the system prompt every conversation. Without size limits, the memory file would grow endlessly, eventually consuming massive amounts of context window. 200 lines x ~125 characters/line is approximately 25KB -- a balance point Anthropic determined through testing as "sufficient but not wasteful."

Four Memory Types

The source code's `memoryTypes.ts` defines four memory types, each with different purposes and storage strategies:

Type	What It Stores	Change Rate	Example
user	User's identity, role, preferences	Very slow	"User is a senior engineer who prefers TypeScript"
feedback	User feedback on AI behavior	Medium	"Don't mock the database in tests"
project	Project status, goals, deadlines	Faster	"API refactor target is April 15 completion"
reference	External system locations and purposes	Slower	"Bug tracker is in Linear's INGEST project"

This categorization isn't arbitrary. Each type corresponds to a different usage strategy. User and feedback memories are useful in almost every conversation (they affect how the AI works). Project memories are only relevant in related contexts. Reference memories are only needed when users mention external systems.

The Core Design Decision: Remember Preferences, Forget Code

The source code has an explicit "what not to store" specification:

```
## What NOT to save in memory

- Code patterns, conventions, architecture, file paths,
  or project structure
- Git history, recent changes, or who-changed-what
- Debugging solutions or fix recipes
- Anything already documented in CLAUDE.md files
- Ephemeral task details
```

Why not store code-related information? A single example makes it clear:

Suppose the AI remembers "the validation function `validateUser` is at line 42 of `src/utils/auth.ts`." Next conversation, you've refactored the code, and that function has moved to line 15 of `src/services/auth/validation.ts`. The AI's memory is now **wrong**, and it doesn't know the memory is stale.

Worse, the AI might start modifying the wrong file based on this incorrect memory. Memory isn't like search. Search always gives you the latest results; memory gives you "what was remembered last time."

So Claude Code's approach: **code-related facts are always retrieved in real time, never memorized.** Memory only stores human preferences and judgments -- things that change slowly and won't become

invalid because of code refactoring.

> **Design Principle**: Persistence should store "things that change slowly." Model capabilities (search, file reading) can retrieve "things that change fast" in real time. Putting fast-changing things into memory is creating a ticking time bomb.

The Extraction Pipeline

Memory extraction doesn't trigger on every message. Rate-limiting logic in the source code ensures extraction doesn't happen too frequently -- only after the AI completes a round of responses.

Extraction work is done by an independent **fork agent**. This sub-AI inherits the main conversation's context (so it knows what was just discussed) but has strictly limited permissions: it can only read files and write to the memory directory -- it can't even run Bash commands.

Why use an independent agent instead of letting the main AI handle it? Two reasons:

Context isolation. The memory extraction process (analyzing the conversation, judging what's worth remembering, formatting memory files) generates lots of intermediate reasoning. Doing this in the main conversation would consume main conversation context window space. The fork agent has its own context, so it doesn't affect the main conversation.

Security isolation. Memory files are persistent and effective across sessions. If the main AI could freely write to memory, malicious prompt injection attacks could achieve persistence through the memory system -- inject malicious memory in one conversation, and it takes effect in all future conversations. Restricting memory writing to a dedicated, minimally-privileged agent shrinks the attack surface.

Cache sharing design. This is a very elegant optimization. The fork agent must share the exact same system prompt, tool list, and model as the main conversation, so both can reuse the same prompt cache. If the fork changed any of these, the cache key would differ, and the fork would need to reprocess the entire prompt -- 10x slower or more.

So the fork agent's permission control isn't implemented by modifying the tool list (which would break the cache) but through a `canUseTool` hook function. The tool list is identical, but the hook intercepts disallowed operations at runtime. This design satisfies both "cache efficiency" and "security isolation" -- two seemingly contradictory requirements.

Bidirectional Mutual Exclusion

The main agent and extraction fork have a mutual exclusion relationship. The source code detects whether the main agent has already directly written to memory files in the current turn:

```
if (hasMemoryWritesSince(messages, lastMemoryMessageUuid)) {
  // Main agent already wrote, skip background extraction
  return
}
```

If the main agent has already saved memory based on its own judgment, background extraction skips this round. This prevents two agents from simultaneously modifying the same memory file.

Stash Mode for Overlapping Requests

If users send multiple messages in rapid succession, a new round might trigger before the previous extraction completes. The source code's approach is **stash-and-continue**: don't start a new extraction, but stash the new request. When the previous round completes, process the stashed request. This prevents extraction task pile-up.

MEMORY.md Index Design

MEMORY.md isn't the memory itself -- it's an **index**. Each memory's detailed content lives in independent files (like `user_role.md`), and MEMORY.md contains only a one-line pointer:

```
- [User Identity](user_role.md) -- Senior engineer, prefers TypeScript
```

This design solves the 200-line constraint problem. The index can point to any number of memory files, each of any length. The 200-line limit on MEMORY.md only constrains "the number of simultaneously visible memory entries," not "total memory capacity."

Each memory file has a standard frontmatter format:

```
---
name: User Role
description: Senior engineer, prefers TypeScript and clean style
type: user
---

Memory content...
```

The `description` field is important -- it's not just a human-readable annotation but is actively used for **memory recall**. When a new session starts, the system uses an independent Sonnet side query (sideQuery) to match all memory file descriptions against the current conversation context, selecting the most relevant ones to load into context.

If the description is too vague (like "some preferences"), it won't match specific scenarios. If too narrow (like "about React component testing"), it's useless in other scenarios. A good description should be specific enough to match yet abstract enough to generalize.

Memory Age Awareness

The source code also has a granular age awareness mechanism. When each memory is recalled, the system calculates its age, and memories older than 1 day receive a staleness warning:

```
export function memoryFreshnessText(mtimeMs: number): string {
  const d = memoryAgeDays(mtimeMs)
  if (d ≤ 1) return ''
  return `This memory is ${d} days old. Memories are
    point-in-time observations, not live state -- claims
    about code behavior or file:line citations may be
    outdated. Verify against current code before
    asserting as fact.`
}
```

Memories within 1 day get no warning; those older than 1 day are explicitly flagged: "This is an N-day-old observation, not current state. Claims about code behavior or file locations may be outdated -- verify before asserting as fact."

This design prevents a subtle problem: the AI might confidently say "this function is on line 42" based on a 3-day-old memory, but the code has been refactored. The age warning makes the AI skeptical when using old memories, rather than blindly trusting them.

autoDream: Tidying Memories in Sleep

There's also a feature called autoDream. Triggering requires passing a **triple gate**, all conditions met: a time gate (at least 24 hours since last consolidation, checked via file mtime), a session gate (at least 5 new sessions accumulated), and a lock gate (no other process currently consolidating, checked via file lock). There's also a 10-minute scan throttle to prevent frequent checking. If a run fails, the system rolls back the lock's mtime to allow retry next round.

The tidying process has four stages:

1. **Orient**

   Review existing memory files to understand the current memory landscape.

2. **Gather Signal**

   Extract information worth remembering from recent session logs.

3. **Consolidate**

   Write new information into persistent memory files. Important detail: relative dates (like "next Thursday") must be converted to absolute dates, otherwise the memory becomes uninterpretable in the future.

4 **Prune / Index**

Remove outdated memories, ensuring MEMORY.md stays within 200 lines / 25KB.

The name "Dream" is evocative -- like how the human brain organizes daytime memories during sleep. autoDream runs silently in the background; users are completely unaware. It uses read-only Bash access and is purely reflective and non-destructive.

Trusting Recalled Memories

The source code contains an important instruction:

```
Memory records can become stale over time. Use memory as
context for what was true at a given point in time. Before
answering the user or building assumptions based solely on
information in memory records, verify that the memory is
still correct and up-to-date by reading the current state
of the files or resources.
```

Translation: **memory is "what was true at a point in time," not "what is true now."** Before using memory for decisions, verify it's still correct.

This solves a subtle problem. If the AI blindly trusts memory, old memories could lead to incorrect behavior. But if the AI completely ignores memory, the memory system has no purpose. The compromise: **treat memory as a lead, not as truth.** Memory says "function X is in file Y"? Go check Y to see if X is still there.

Lessons for AI Product Builders

> KEY TAKEAWAY
>
> **The simplest storage solution is often the best.** Plain text files, Markdown format, file system directory structure. No database, no API, no complex query language. This solution has near-zero maintenance cost, and users can directly view and edit their own memory files with a text editor.

Categorized storage, tiered trust. Different types of memory have different change rates and usage frequencies. User preferences rarely change and should be considered every time. Project status changes quickly and is only useful in relevant contexts. One-size-fits-all "remember everything" or "remember nothing" is inferior to categorized management.

Set limits on the memory system. 200 lines, 25KB. A memory system without limits will inflate to unmanageable levels. Limits force the system to make tradeoffs, keeping only truly valuable memories and discarding trivial details.

Use an independent agent for memory extraction. Security isolation + context isolation + cache sharing. These three requirements seem contradictory, but through the canUseTool hook function, they're elegantly unified.

Record successes, not just failures. Feedback-type memory has a counterintuitive requirement: record not just user corrections ("don't do this") but also user validations ("yes, exactly like that"). The reason: if only failures are recorded, the AI becomes overly conservative, afraid to do anything. Recording successes lets the AI know which approaches are approved and can be continued in similar scenarios.

Time normalization is essential. When a user says "next Thursday," the AI must store it in memory as "2026-04-10." Otherwise, two weeks later when reviewing this memory, which Thursday does "next Thursday" refer to? This rule seems trivial, but it solves an ambiguity problem when recalling memories across time.

Even when users ask to remember, probe further. The source code has an interesting rule: when a user says "remember this week's PR list," the AI shouldn't directly memorize the PR list (that's ephemeral information) but should ask "what about it was unexpected or non-obvious?" **That's** what's worth remembering. This prevents the memory system from being used as a notebook.

07 Context Management: Surviving Long Conversations

Context Management — Surviving Long Conversations

After a long conversation with Claude, it occasionally forgets a requirement you mentioned earlier. This isn't it being dismissive -- it's the inevitable cost of context compression. The source code reveals that Anthropic made very deliberate design choices about "what to preserve."

When Compression Triggers

The source code has two compression paths:

Auto compression (autoCompact): Triggered automatically when the conversation's token count approaches the context window limit. The source code defines precise trigger thresholds:

```
AUTOCOMPACT_BUFFER_TOKENS = 13_000

autoCompactThreshold = effectiveContextWindow - 13_000
```

Meaning: auto-compression triggers when token usage reaches "effective context window - 13,000." For a 200K context window, that's approximately 187K. The 13,000-token buffer gives the system enough space to complete the compression operation itself.

Manual compression: Triggered when users execute the `/compact` command. Typically used when the conversation feels slow or you want to clean up context.

There's also a **micro compact**, a lighter compression method implemented in `microCompact.ts`. It doesn't compress the entire conversation -- only the return results of tool calls (e.g., a grep search that returned hundreds of lines of code gets compressed to keep only key findings).

The source code also mentions two additional compression modes: **reactive compaction** (triggered when the API returns a "context too long" error) and **context folding** (an experimental more efficient compression method). These modes are mutually exclusive with auto-compression; only one is active at a time.

The Nine-Section Structured Compression

Compression isn't just casually "summarizing." The source code's `compact/prompt.ts` defines a strict nine-section structured extraction template. The model must output compression results in the following format:

Section	Content	Importance
1. Primary Request and Intent	All of the user's explicit requests and intent	Highest
2. Key Technical Concepts	Technical concepts, frameworks, technologies discussed	High
3. Files and Code Sections	Specific filenames, code snippets, modification records	High
4. Errors and Fixes	Errors encountered and fix methods	Medium-High
5. Problem Solving	Problem-solving process and ongoing investigation	Medium
6. All User Messages	All user messages that are not tool results	Highest
7. Pending Tasks	Tasks explicitly requested but not yet completed	High
8. Current Work	Detailed description of work in progress at compression time	Highest
9. Optional Next Step	Next step plan (must directly relate to user's most recent request)	Medium

Why All User Messages Must Be Preserved

Section 6 is the most critical design decision: **all user messages must be listed in full.**

The source code prompt explicitly requires: "List ALL user messages that are not tool results. These are critical for understanding the users' feedback and changing intent."

Why? Because every message from the user may contain implicit preferences and corrections.

For example: in round 3, the user said "don't use var, use const." If compression drops this message, the AI might start using var again in subsequent code. The user thinks the AI has already learned, but in reality that correction was "forgotten."

User messages are the most damaging part to lose in compression, because they're not just information -- they're constraints.

Two-Stage Compression: Analysis + Summary

The compression process has two stages:

Analysis stage: The model first does free-form analytical thinking within `<analysis>` tags, ensuring all key points are covered. This stage's output is not retained in the final summary.

Summary stage: Based on the analysis results, outputs a structured summary in the nine-section format. This summary becomes the initial context for the next conversation round.

Why two stages? Because directly asking the model to output a structured summary tends to miss details. The analysis stage lets the model "think freely" first, ensuring nothing is missed, before formatting the output.

Source code comments explain a specific engineering problem:

```
// Aggressive no-tools preamble. On Sonnet 4.6+ adaptive-thinking
// models the model sometimes attempts a tool call despite the
// weaker trailer instruction. With maxTurns: 1, a denied tool
// call means no text output → falls through to the streaming
// fallback (2.79% on 4.6 vs 0.01% on 4.5).
```

Translation: during compression, the model has only 1 turn to output. If it attempts a tool call (which gets denied), there's no text output, and compression fails. Sonnet 4.6's failure rate is 2.79% (versus 4.5's 0.01%), so they added an aggressive "no tool calls" preamble to reduce the failure rate.

This detail exposes a real engineering challenge: **different model versions require different prompt strategies.** Model upgrades aren't always a free lunch -- new versions can introduce new problems.

Anti-Drift Design in Section 9

Section 9 (Next Step) has a lengthy warning:

```
IMPORTANT: ensure that this step is DIRECTLY in line with
the user's most recent explicit requests. Do not start on
tangential requests or really old requests that were already
completed without confirming with the user first.

Include direct quotes from the most recent conversation
showing exactly what task you were working on and where
you left off. This should be verbatim to ensure there's
no drift in task interpretation.
```

This rule addresses the **task drift** problem. After compression, the model might start working on an old task mentioned in the summary rather than continuing what the user most recently requested.

Requiring "verbatim quotes" from the original conversation anchors the model to the actual user request rather than its interpretation of the request.

Craftsmanship Behind the Compact Prompt

We've seen the skeleton of the nine-section structure. Now let's look at several easily overlooked refinements in the prompt text.

Why Section 8 Requires "Direct Quotes"

The prompt text for Section 8 (Current Work) and Section 9 (Next Step) contains wording worth examining word by word:

```
Include direct quotes from the most recent conversation
showing exactly what task you were working on and where
you left off. This should be verbatim to ensure there's
no drift in task interpretation.
```

Note the word choices: not "describe," not "summarize," but **"direct quotes"** and **"verbatim"** (word-for-word copy).

Why insist on quoting the original text? Because every compression is a "translation" -- converting a long conversation into a short summary. Translation loses nuance. The user says "make the button a bit smaller," and if compressed to "adjust button styling," the model might change the color too. Requiring verbatim quotes anchors the model's understanding with original text, preventing each compression from drifting a little until, after several compressions, things go completely off track.

This problem is called **task drift**, and it's a core challenge for all long-conversation AI systems. Anthropic's solution isn't a more complex algorithm but a simple prompt rule: make the model copy the original text. Sometimes the most effective engineering solution is the most straightforward one.

Partial Compact vs Full Compact

The source code has two compression modes:

Dimension	Full Compact	Partial Compact
Scope	Entire conversation history	Only compresses some messages, preserves rest as-is
Trigger	Auto-compression, manual /compact	Intelligent split-point selection
Prompt Cache	Fully invalidated	Can preserve first-half cache when direction=from
Information Loss	Higher	Lower (preserves some original messages)

Partial Compact has two directions:

direction = 'from': Preserves older messages as-is, only compresses newer messages. The older messages' prompt cache hits fully -- no reprocessing cost.

direction = 'up_to': Compresses older messages, preserves newer messages as-is. Prompt cache is invalidated (because the compression summary is inserted at the front), but newer messages maintain full context.

The Partial Compact prompt has subtle differences too. Full Compact's prompt says "create a detailed summary of **the conversation**," while Partial Compact says "summary of **the RECENT portion** of the conversation -- the messages that follow earlier retained context." This isn't casual wording. It tells the model: earlier messages are still there, you only need to summarize the new portion, don't repeat existing context.

Micro Compact: Surgical Compression of Tool Results

The Full Compact and Partial Compact discussed above are major surgery on the entire conversation. But Claude Code also has a more precise "microsurgery": Micro Compact, defined in `microCompact.ts`.

What Problem It Solves

In practice, tool return results are often very large. A single grep search might return hundreds of lines of code; a file read might return thousands. These tool results are useful at the time, but 5 turns later, they become "dead weight" in the context -- occupying space without providing new information.

Micro Compact doesn't compress the entire conversation. It only cleans up individual tool return results, replacing old results with a brief placeholder:

```
const TIME_BASED_MC_CLEARED_MESSAGE = '[Old tool result content cleared]'
```

Which Tools Get Compressed

Not all tool results get cleaned up. The source code defines an allowlist:

```
const COMPACTABLE_TOOLS = new Set([
  FILE_READ_TOOL_NAME,     // File read
  ...SHELL_TOOL_NAMES,     // Shell commands
  GREP_TOOL_NAME,          // Search
  GLOB_TOOL_NAME,          // File discovery
  WEB_SEARCH_TOOL_NAME,    // Web search
  WEB_FETCH_TOOL_NAME,     // Web fetch
  FILE_EDIT_TOOL_NAME,     // File edit
  FILE_WRITE_TOOL_NAME,    // File write
])
```

These tools share a common trait: their return results can be very large, but the core information has already been digested by the model. The model read a 1,000-line file and extracted key information -- the 1,000-line original no longer needs to occupy context.

Two Trigger Methods

Time-based trigger: If the user has been away for a while (e.g., time since the last conversation has passed), the server-side prompt cache has expired, and the entire prefix needs reprocessing. At that point, it's efficient to also clean up old tool results, reducing tokens to reprocess.

Cache-edit-based trigger (Cached MC): A more advanced method that uses the API's cache_edits feature to delete tool results directly from the server-side cache, without resending complete messages. This saves tokens without breaking existing cache.

> KEY TAKEAWAY
>
> **Micro Compact's design philosophy is worth learning.** Rather than waiting until context is nearly full to perform major surgery (Full Compact), do continuous small cleanups in the meantime. This is similar to "incremental GC" in memory management -- frequent small cleanups have less system impact than occasional large ones. If you're building a long-conversation AI application, consider incremental tool result cleanup before full compression.

The Cost of Compression: Information Will Be Lost

After understanding the compression mechanism, one fact must be faced: **no matter how sophisticated the compression strategy, information loss is inevitable.**

Compression's essence is representing longer conversation content with shorter text. No matter how good the nine-section structure is, it only enables the model to "selectively forget" rather than "not forget."

What Information Is Most Easily Lost

Information Type	Loss Risk	Reason
Implicit preferences	High	One-liners like "don't use var" are easily compressed away
Attempted-but-failed approaches	High	Errors and Fixes section preserves errors, but attempted approach details are easily lost
Discussion process	High	Discussed options A, B, C and chose B; after compression, might only have "chose B"
Specific code modifications	Medium	Section 3 preserves files and code, but early modifications in long sessions may get overwritten
User's direct requests	Low	Section 6 requires full preservation of all user messages -- this is the best-protected part
Current work in progress	Low	Section 8 requires verbatim quotes -- strong protection

How to Reduce Information Loss from Compression

Understanding the compression mechanism lets you turn it to your advantage:

1. Write critical constraints in CLAUDE.md. CLAUDE.md is in the system prompt -- compression doesn't touch it. If you have a rule that needs to hold for the entire project (like "use TypeScript" or "name functions in camelCase"), writing it in CLAUDE.md is far more effective than repeatedly mentioning it in conversation. What you say in conversation gets compressed; what's written in CLAUDE.md stays forever.

2. Periodically restate key requirements in long sessions. If you notice the AI starting to "forget" a rule, don't get frustrated. It probably just went through an auto-compression, and the original context for that rule has been replaced by a summary. Just say it again.

3. Use /compact proactively rather than waiting for auto-trigger. Manual compression lets you attach custom instructions via the `/compact` command, telling the model which information to specially preserve during compression. Auto-compression doesn't give you this opportunity.

4. Break large tasks into multiple sessions. Compression is a stopgap within a single session. If a task is complex enough to need multiple compressions, it's better to split into multiple sessions, with each session's CLAUDE.md clearly specifying the context. This is more reliable than having the model work on summaries of summaries.

5. Pay attention to the first reply after /compact. The first reply after compression is most likely to have issues. If the AI's understanding seems off, correct it immediately. This correction becomes new context after the compression summary, influencing all subsequent conversation.

Lessons for AI Product Builders

> **KEY TAKEAWAY**
>
> **Structured compression is far more reliable than free-form summarization.** If you tell the AI "summarize the previous conversation," it may omit critical details. Give it an explicit template -- which information must be preserved, in what format -- and results improve dramatically. The nine-section template is this principle put into practice.

User messages are the information you can least afford to lose. Data from tools can be re-queried, code can be re-read, but what users have said represents intent and constraints. Losing user messages means losing alignment.

Anti-drift requires proactive design. Every compression can introduce small understanding deviations that accumulate over multiple compressions. Requiring verbatim quotes and constraining next steps to relate to the most recent request are proactive anti-drift measures.

Model upgrades require prompt adaptation. The failure rate difference between 4.5 and 4.6 (0.01% vs 2.79%) shows that model version changes can break existing prompt strategies. Continuously monitoring model behavior and promptly adjusting prompts is a long-term engineering task.

08 Search: Why grep Beats RAG

Why grep Beats RAG

This might be the most counterintuitive finding in the entire source code. While the industry pushes toward vector databases and embedding search, Anthropic chose the most basic approach possible.

What You'd Expect

If you were building a code search feature for an AI coding tool in 2026, the industry-standard approach would look something like this:

1. Chunk the codebase into small pieces
2. Use an embedding model to turn each chunk into a vector
3. Store them in a vector database (Pinecone, Weaviate, Chroma, etc.)
4. When users query, convert the query to a vector and do similarity search
5. Feed the most relevant code chunks to the LLM

That's RAG (Retrieval-Augmented Generation). The hottest architectural paradigm in AI application development. Countless startups are building RAG-related tools and infrastructure.

What Claude Code Actually Does

grep. And ripgrep.

No embeddings. No vector database. No semantic search. Just plain text pattern matching.

The search-related tools in the source code are just two: `Grep` (ripgrep-based content search) and `Glob` (filename pattern matching). Plus the `Read` tool for directly reading file contents. That's it.

You think it searches code accurately? Behind the scenes, it's just grep plus a sufficiently smart brain.

Why This Works Better

The logic behind this decision is quite clear -- once you think it through, it feels obvious:

1. The LLM Is Already Smart Enough

The core problem RAG solves is: search engines aren't smart enough and need semantic understanding to find relevant content. But when the consumer of search results is a world-class LLM,

the situation changes.

Claude Opus can understand relationships between code from a pile of grep results, infer call chains, and identify design patterns. It doesn't need a smart search engine to pre-filter and rank -- it is the strongest "understanding engine."

Rather than making every component complex, keep one component powerful enough and the others simple.

2. grep's Advantages Are Underestimated

grep does one thing and does it extremely well: precise text matching. It tells you "this string appears in these files at these lines," 100% accurate, no false positives, virtually no latency.

Compare with RAG's embedding search: results are probabilistic, possibly returning "semantically related but actually irrelevant" content, or missing "differently written but actually related" content. Plus you need to tune chunk size, choose embedding models, maintain vector indices. Each step can go wrong.

An analogy: grep is like looking up books by ISBN in a library -- precise but requires knowing the ISBN. RAG is like asking someone to find "books roughly about this topic" -- flexible but might grab the wrong ones. When you have a smart enough librarian (LLM) who knows which ISBNs to look up, the former's precision becomes more valuable.

3. The Maintenance Cost Gap Is Enormous

grep requires zero maintenance. Code changed? grep searches the latest content directly. No index to update, no vectors to rebuild, no database to manage.

RAG's maintenance costs aren't trivial: every code change requires re-chunking and re-embedding, vector indices can become stale, embedding model upgrades require rebuilding all vectors, and the database itself needs ops. For a tool that switches between dozens of projects daily, this maintenance cost is catastrophic.

Claude Code needs to "open any project and just work," making zero-maintenance grep the only viable approach. If every new project required building an index first, user experience would suffer greatly.

4. Context Windows Got Bigger

RAG was essential in the era of small context windows (4K, 8K) -- you could only fit so much into the prompt and had to pick carefully. But Claude Code uses 100K or even 1M token context windows.

With larger windows, brute-force search becomes viable. Search a few more files, read a few hundred more lines of code -- there's room for it. No need for precision retrieval to save every token.

Inside the Grep Tool

Having covered "why not RAG," let's see how Anthropic actually designed the Grep tool. The source code's GrepTool is based on ripgrep (rg), not GNU grep. This choice itself is telling. ripgrep is an order of magnitude faster than traditional grep, natively supports .gitignore, Unicode, multithreading, and is heavily optimized for large codebases.

The GrepTool input parameter definition is information-rich:

```
const inputSchema = lazySchema(() =>
  z.strictObject({
    pattern: z.string().describe(
      'The regular expression pattern to search for in file contents',
    ),
    path: z.string().optional(),
    glob: z.string().optional().describe(
      'Glob pattern to filter files (e.g. "*.js", "*.{ts,tsx}")'
    ),
    output_mode: z.enum([
      'content', 'files_with_matches', 'count'
    ]).optional(),
    '-B': semanticNumber(z.number().optional()),  // Before context lines
    '-A': semanticNumber(z.number().optional()),  // After context lines
    '-C': semanticNumber(z.number().optional()),  // Symmetric context
    '-n': semanticBoolean(z.boolean().optional()), // Line numbers
    '-i': semanticBoolean(z.boolean().optional()), // Case insensitive
    type: z.string().optional(),  // File type filter
    head_limit: semanticNumber(z.number().optional()),
    offset: semanticNumber(z.number().optional()),
    multiline: semanticBoolean(z.boolean().optional()),
  })
)
```

The three `output_mode`s are cleverly designed:

`files_with_matches` (default) -- returns only matching file path lists. The most token-efficient mode, suitable for a first-pass search to "identify which files are relevant."

`content` -- returns matching lines with context. Supports `-A` / `-B` / `-C` parameters for context line control and `-n` for line numbers. Suitable for "I know which file, I want to see specific code."

`count` -- returns only match counts per file. Suitable for statistical scenarios like "how many times is this function called."

These three modes are essentially an **information density gradient**. The LLM can first use `files_with_matches` for low-cost scanning, then `content` for precise reading after narrowing targets. This is far more efficient than returning everything at once.

There's also an understated but important design: `head_limit` defaults to 250 lines.

```
// Default cap on grep results when head_limit is unspecified.
// Unbounded content-mode greps can fill up to the 20KB persist
// threshold (~6-24K tokens/grep-heavy session).
// 250 is generous enough for exploratory searches while preventing
// context bloat. Pass head_limit=0 explicitly for unlimited.
const DEFAULT_HEAD_LIMIT = 250
```

This comment is practically a mini-paper. Unlimited grep results could return tens of thousands of lines, directly consuming the context window. 250 lines is a balance point: enough for initial exploration, but not token-wasteful. If the LLM truly needs more results, it can explicitly pass `head_limit=0` or use `offset` for pagination.

Why not just let the AI call `bash grep`? The BashTool prompt explicitly requires **avoiding the grep command** in favor of the Grep tool. Reasons include: GrepTool has permission checking (`checkReadPermissionForTool`), ensuring files that shouldn't be searched aren't; it automatically excludes `.git`, `.svn`, and other version control directory noise; it limits each line to a maximum of 500 characters (`--max-columns 500`), preventing base64 or minified code from bloating results; and it converts absolute paths to relative paths to save tokens. Raw grep can't do any of this.

GlobTool: File Discovery First

Grep handles "what to search in files"; Glob handles "find which files first." This is the first step in the search flow, often overlooked but critically important.

The Glob tool's backend is also ripgrep, using `rg --files --glob` mode to list files matching the pattern rather than searching file contents. It inherits all of ripgrep's performance advantages.

A key design detail: **results are sorted by modification time**.

```
const args = [
  '--files',
  '--glob',
  searchPattern,
  '--sort=modified',  // Sort by modification time (oldest first)
  ...(noIgnore ? ['--no-ignore'] : []),
  ...(hidden ? ['--hidden'] : []),
]
```

Glob results are sorted by modification time (ripgrep's `--sort=modified` is ascending, oldest first), limited to 100 files. The LLM gets the complete file list and can judge relevance based on context.

GrepTool's `files_with_matches` mode goes further with application-layer optimization, sorting in descending order so **most recently modified files appear first**:

```
// files_with_matches mode
const stats = await Promise.allSettled(
  results.map(_ => getFsImplementation().stat(_)),
)
const sortedMatches = results
  .map((_, i) => {
    const r = stats[i]!
    return [
      _,
      r.status === 'fulfilled' ? (r.value.mtimeMs ?? 0) : 0,
    ] as const
  })
  .sort((a, b) => b[1] - a[1])  // Most recently modified first
  .map(_ => _[0])
```

This isn't a technically complex decision, but it reflects product thinking: **the ordering of search results is itself a form of "intelligence."** No vector database needed to judge relevance -- modification time is an extremely strong signal. Grep putting recently modified files first in `files_with_matches` mode, Glob sorting by time and capping at 100 files -- combined, they efficiently zero in on targets.

The Read Tool: Underrated Search Weapon

The Read tool looks like "file reading" on the surface, but it's actually the last link in the search system and its most underrated component.

`offset` and `limit` parameters mean the LLM doesn't need to read entire files. It can first use Grep to find `validateUser` on line 187, then use `Read(file, offset=180, limit=30)` to read only the surrounding code. For files with thousands of lines, this saves enormous amounts of context.

But Read's truly surprising capability is multimodal. It can read images (leveraging the LLM's multimodal ability to directly "see" screenshots and design mockups), read PDFs page by page, and parse Jupyter Notebooks (including outputs and visualizations). Combined, Read isn't just a file reader -- it's a complete **document understanding tool**.

There's also an optimization hidden deep in the code: `FILE_UNCHANGED_STUB`. If the LLM repeatedly reads the same file in the same session and the file hasn't changed, Read returns a brief note instead of the full contents:

```
export const FILE_UNCHANGED_STUB =
  'File unchanged since last read. The content from
   the earlier Read tool_result in this conversation
   is still current -- refer to that instead of
   re-reading.'
```

Another token-saving strategy. The LLM often re-reads the same file during debugging; this stub avoids redundant loading.

Viewing Grep + Glob + Read together: **Glob discovers files → Grep locates positions → Read precisely reads**. Three tools, each doing one thing, combining into a complete code understanding system. No index, no preprocessing, ready to use instantly.

No Vector Database -- The Evidence

Earlier I said Claude Code doesn't use a vector database. This isn't speculation -- it's empirical.

I searched all 512,920 lines of Claude Code source code for the following keywords:

`embedding`: Found 2 matches -- one for "embedding a file:// URL" (embedding a link in a URL), one for "string-embedding" (embedding strings in bash commands). Both are ordinary English word usage, completely unrelated to vector embedding.

`vector`: 0 matches. No vector database, no vector search, no vector index.

`semantic search`: 0 matches.

`pinecone / weaviate / chroma / faiss`: 0 matches. No references to any mainstream vector database.

510,000 lines of code. Zero vector search.

This isn't "haven't gotten to it yet" or "will add in the next version." Claude Code is already one of the most successful AI coding tools on the market, and in this state, it has proven: for the code search use case, you don't need a vector database.

As an AI company, Anthropic understands the capabilities and limits of embeddings and RAG better than anyone. They chose not to use them -- an **intentional architectural decision**, not a capability gap. The entire search system is just three tools, under 1,500 lines of code, yet it powers Claude Code's ability to understand codebases of any scale.

This serves as a reminder for everyone building AI applications: **when you have a hammer (RAG), not every problem is a nail.** In an era where LLMs are powerful enough, the simplest toolchain is often the most effective.

How the Search Workflow Works

Claude Code's typical code search flow looks like this:

1. **Understand Intent**

   The LLM understands what the user wants to find. For "user data validation logic," it knows to search keywords like `validate`, `user`, `schema`.

2. **Choose Search Strategy**

   First use Glob to find relevant files (`**/*user*.ts`), then use Grep within candidate files to search for keywords (`validate`). May run multiple rounds with different searches to cover various naming conventions.

3. **Read and Understand**

   After finding candidate files, use the Read tool to get full contents. The LLM understands code meaning, call relationships, and design intent.

4. **Iterative Search**

   If the first round doesn't find it, the LLM adjusts search strategy based on existing findings. For example, discovering that validation logic is in the `middleware` directory rather than `models`, it redirects there.

The bottleneck for search quality isn't in the retrieval algorithm -- it's in search strategy. Knowing what to search, where to search, and how to refine direction from initial results -- these are what LLMs excel at. grep gives it precise raw data; the LLM handles all higher-level intelligence. Clear division of labor.

Lessons for AI Product Builders

> KEY TAKEAWAY
>
> **The "Good Enough" Principle**: When designing AI applications, don't default to needing the most advanced technology for every component. First ask: if this component uses the simplest approach, where's the bottleneck? If the LLM's own capability can compensate for the simple approach's shortcomings, then use the simple approach. The maintenance cost of complex solutions may outweigh the intelligence they save.

A few generalizable criteria:

If the LLM is the consumer, search can be simple. RAG is suited for scenarios where "search results are displayed directly to humans." When search results are intermediate steps for the LLM, precise matching + LLM understanding > fuzzy matching + pre-ranking.

Zero maintenance beats high accuracy. If your tool needs to work instantly in any environment (like Claude Code), any approach requiring preprocessing is an obstacle.

Subtract first, then consider adding. When you want to add a complex component, first ask: would it work without it? If yes, don't add it. Claude Code proved that you can build the best AI coding tool without a vector database.

Of course, this conclusion has boundary conditions. For enterprise knowledge bases with millions of documents, pure grep might not suffice. But for the specific scenario of code search, projects typically range from tens of thousands to hundreds of thousands of lines -- grep is fast enough, the LLM's comprehension is strong enough, and the simple solution is the optimal solution.

09 Multi-Agent Architecture: Run Like a Company

Multi-Agent Architecture — Run Like a Company

Claude Code's multi-agent system is arguably the most complex part of the entire source code. It's not just forking a few processes to run in parallel -- it implements a complete organizational management architecture.

Organization Structure

The source code's `src/utils/swarm/` directory implements a multi-agent collaboration framework. The core organizational model:

Role	Responsibility	Permissions
Team	Container for a group of agents, manages resources and objectives	Highest
Leader	Assigns tasks, approves permission requests, merges results	High
Teammate	Executes specific tasks, reports progress	Restricted

This mirrors a real company's structure almost exactly. The Leader is the tech lead, Teammates are developers, and the Team is the project group.

Three Execution Modes

Agents can run in parallel through three methods:

In-process isolation: Agents run within the same Node.js process, with context isolation via `AsyncLocalStorage`. Each agent has an independent AbortController (a Teammate isn't interrupted when the Leader is), independent state storage, and shared API client and MCP connections. Lightest weight, zero spawn overhead, suitable for simple parallel search tasks.

tmux windows: Each agent runs in a separate tmux pane. You can see each agent's real-time output. Suitable for scenarios requiring progress monitoring.

iTerm2 split panes: Uses iTerm2's split-screen capability, with each agent in an independent pane. Most visually intuitive, but only available on macOS + iTerm2.

Why three methods? Different use cases have different visibility needs. Simple research doesn't need to see every agent's output, but complex multi-module development benefits from seeing all agents'

progress simultaneously.

Isolation via Git Worktree

This is the most critical design decision in the multi-agent architecture.

When multiple agents modify code simultaneously, the biggest risk is **conflicts**. Agent A is editing file X, Agent B is also editing file X -- they overwrite each other.

Claude Code's solution is `git worktree`. Each agent works in its own independent working tree with an independent file system copy. After completion, results are merged through git's merge mechanism.

Git worktree isn't a complete repo clone. It shares the same `.git` directory, just creating a new working directory and branch. Fast to create, small disk footprint, suited for short-lived agent tasks.

If an agent makes no changes, the worktree is automatically cleaned up. If changes were made, the worktree path and branch name are returned to the caller for subsequent merging.

Mailbox Communication

How do agents communicate? Not through API calls, not through message queues. Through **mailbox files**.

Each agent has a mailbox file. Other agents wanting to send it a message write to this file. The agent periodically checks its mailbox, reading new messages.

This is another example of the "simplest possible approach." The file system is the most reliable communication infrastructure -- no extra services to start, no connection timeouts, messages survive restarts. The downside is higher latency (periodic polling), but for agent-level collaboration (minute-scale tasks), a few seconds of delay is perfectly acceptable.

Permission Bubbling

When a Teammate encounters an operation requiring confirmation (like deleting files), the permission request doesn't go directly to the user. It **bubbles up** to the Leader. The Leader decides whether to approve.

Why not ask the user directly? Because if 5 agents are working simultaneously, each popping up permission confirmation dialogs, the user would be overwhelmed. The Leader serves as a middle layer that can batch-process permission requests based on task context, reducing interruptions.

This is identical to real company approval workflows. Developers don't ask the CEO to confirm every operation -- they let the tech lead make the call.

The Coordinator Pattern

Beyond the basic Team mode, Claude Code also implements a more advanced **Coordinator Mode**. Four phases:

1. **Research**

   Multiple workers investigate the codebase in parallel, gathering information. For example, one worker analyzes frontend code while another analyzes the backend API.

2. **Synthesis**

   The Coordinator collects all workers' findings, synthesizes the analysis, and generates a specification. This step is crucial -- it transforms scattered information into a unified action plan.

3. **Implementation**

   Workers make precise modifications according to the specification. Each worker handles only their part, following the unified spec.

4. **Verification**

   Workers verify modification results. This may include running tests, checking compatibility, confirming no new issues were introduced.

Dissecting the Coordinator Prompt

The Coordinator Mode's system prompt is itself an extremely valuable document. The source code's role description gets straight to the point:

```
You are a **coordinator**. Your job is to:
- Help the user achieve their goal
- Direct workers to research, implement and verify code changes
- Synthesize results and communicate with the user
- Answer questions directly when possible --
  don't delegate work that you can handle without tools
```

The last point is key: the Coordinator isn't a pure dispatcher -- it should **answer what it can answer directly**. This avoids over-delegation. If a user asks "what does this function do," the Coordinator shouldn't spawn a worker to search -- it should just answer.

The system prompt also has an easily overlooked but crucial rule:

```
Every message you send is to the user. Worker results
and system notifications are internal signals, not
conversation partners -- never thank or acknowledge them.
Summarize new information for the user as it arrives.
```

Workers' results arrive as `<task-notification>` format inserted into the conversation, looking like user messages but they're not. The Coordinator must distinguish between the two: real user messages need responses, while worker notifications are internal signals to be digested, synthesized, and reported to the user in natural language.

There's an even deeper design constraint: **Workers can't see the conversation between Coordinator and user**. Every prompt sent to a Worker must be self-contained, including all context, file paths, line numbers, and specific requirements the Worker needs. This differs from what many people imagine about multi-agent systems. Many assume agents share a single conversation context, but in reality each Worker is an independent world -- it only sees the instruction the Coordinator gave it.

This design choice was deliberate. Shared context sounds convenient but creates two problems: context inflation (each Worker loads the full conversation history) and information interference (previous Workers' exploratory noise affects the next Worker's judgment). Self-contained prompts are cleaner and more controllable.

Permission Sync with File Locks

The permission bubbling concept was covered earlier, but implementation details deserve expansion. The source code's `permissionSync.ts` implements a file-system-based permission request queue.

Directory structure:

```
~/.claude/teams/{teamName}/permissions/
├── pending/          # Pending permission requests
│   ├── .lock         # File-level lock
│   ├── perm-1712345-abc.json
│   └── perm-1712346-def.json
└── resolved/         # Processed permission requests
    └── perm-1712345-abc.json
```

The flow: when a Worker encounters an operation needing permission, it creates a `SwarmPermissionRequest` object (containing toolName, input, description, and other complete information) and writes it to the `pending/` directory. The Leader periodically scans `pending/`, discovers new requests, presents them to the user, the user decides, and the Leader moves the request to `resolved/`. The Worker polls `resolved/`, reads the result, and continues execution.

The source code also has a newer **mailbox-based permission communication** (`sendPermissionRequestViaMailbox`) that uses the mailbox files mentioned earlier, gradually replacing

the file directory approach. The new method reuses existing mailbox infrastructure, avoiding the overhead of separately maintaining pending/resolved directories.

Resolved requests don't accumulate indefinitely. `cleanupOldResolutions()` cleans up resolved files older than 1 hour (`maxAgeMs = 3600000`). A typical engineering tradeoff: keeping too long wastes disk, deleting too soon risks Workers not reading the result. One hour is a reasonable middle ground.

Team Lifecycle Management

Teams aren't just create-and-forget -- their destruction requires careful design too. The source code's `cleanupSessionTeams()` defines a strict cleanup order:

1. **Kill Panes**

   First kill all pane-backed teammate processes. Calls each backend's (tmux/iTerm2) `killPane()` method, ensuring no orphaned processes continue running.

2. **Destroy Worktrees**

   Destroy all members' git worktrees. Reads the team file for worktree path list, then calls `git worktree remove --force` for each. If the git command fails, falls back to direct `rm -rf`.

3. **Clean Up Team Directory**

   Deletes `~/.claude/teams/{team-name}/` and all contents (including permission request files, team configs, etc.).

4. **Clean Up Tasks Directory**

   Deletes `~/.claude/tasks/{taskListId}/` and triggers `notifyTasksUpdated()` to update the UI.

Why does order matter? The source code comment is clear:

```
Kill panes first -- on SIGINT the teammate processes
are still running; deleting directories alone would
orphan them in open tmux/iTerm2 panes.
```

If you delete directories before killing processes, agent processes are still running but their working directories are gone. These orphan processes endlessly error in tmux panes with no one to clean them up. Killing processes before deleting directories prevents this state.

Anti-Laziness Rules

The Coordinator Mode prompt includes strict anti-laziness instructions. The source code explicitly prohibits patterns like:

```
Never write "based on your findings" or "based on the
research." These phrases delegate understanding to the
worker instead of doing it yourself.
```

After receiving a Worker's research report, the Coordinator **must understand the content itself**, then write instructions containing specific file paths, line numbers, and modification plans. It can't pass the understanding work to the next Worker.

Verification also has strict requirements: "Verification means proving the code works, not confirming it exists." Verifiers must actually run tests, check for type errors, and test edge cases. They can't just glance at code and say "looks fine."

Continue vs Spawn Decision Matrix

The Coordinator Mode prompt details when to continue with the same Worker (SendMessage) versus when to spawn a new one:

Scenario	Choice	Reason
Research explored the exact files to edit	Continue	Worker already has files in context
Research was broad but implementation is narrow	Spawn fresh	Avoid exploratory noise interfering with precise implementation
First implementation was completely wrong	Spawn fresh	Wrong approach's context would anchor retries
Verifying another Worker's code	Spawn fresh	Verifier should see code with fresh eyes

.worktreeinclude Support

Worktree isolation has an edge case: some files are gitignored (like `.env` config files, local keys) but needed by agents during work. The source code implements a `.worktreeinclude` file to solve this, listing gitignored files that need to be copied from the main repo to the worktree.

Lessons for AI Product Builders

> **KEY TAKEAWAY**
>
> **The hard part of multi-agent systems isn't technical implementation -- it's organizational design.** How to split tasks, how to flow information, how to resolve conflicts, how to merge results, how to ensure quality -- these problems are identical to managing real human teams.

Isolation over sharing. Git worktree gives each agent an independent file system, fundamentally eliminating concurrent modification conflicts. The tradeoff is an extra merge step, but that's far simpler than handling runtime conflicts.

Use the simplest possible communication scheme. Mailbox files are 100x simpler than message queues, and perfectly adequate for agent-level collaboration. Don't over-invest in communication infrastructure.

A middle management layer is needed. The Leader role isn't redundant. It handles permission bubbling, supervises task progress, and coordinates result merging. Without this layer, 5 agents interacting directly with the user would be chaos.

10 The Future in Feature Flags

The Future in Feature Flags

The source code contains 44 feature flags controlling a batch of unreleased features. These flags are like a window, letting us glimpse the Claude Code team's product roadmap.

GrowthBook: The Feature Flag Infrastructure

Before looking at specific flags, let's see what Anthropic uses to manage them.

The answer is **GrowthBook**, an open-source feature flag and A/B testing platform. Claude Code's source code has a complete GrowthBook integration layer (`src/services/analytics/growthbook.ts`), over 400 lines of code dedicated to feature flag fetching, caching, and experiment tracking.

Key designs:

- **Remote evaluation mode** (remoteEval): Flag values aren't computed on the client but pre-computed by the GrowthBook server based on user attributes and delivered directly. The client receives results, not rules
- **Disk caching**: Flag values are written to the local `~/.claude.json` config file. This means even if the GrowthBook service is temporarily unavailable, Claude Code can operate normally using the last cached values
- **Multi-layer overrides**: Environment variable overrides > local config overrides > remote values. Internal employees can force-specify any flag's value via `CLAUDE_INTERNAL_FC_OVERRIDES` environment variable for debugging and testing
- **Exposure logging**: Each feature flag access records an "exposure event" for subsequent A/B test analysis. Auto-deduplication within the same session prevents hot paths (like flags called repeatedly in render loops) from generating duplicate data

All flags are prefixed with `tengu_`. Tengu is Claude Code's internal project codename.

Why does an AI coding tool need such heavy feature flag infrastructure? Because AI product behavior is far more complex than traditional software. Traditional software behavior is determined by code -- code doesn't change, behavior doesn't change. AI product behavior also depends on model versions, prompt wording, and context management strategies -- any of these changing can cause significant user experience fluctuations. Feature flags let the team **adjust these factors without deploying code**, with instant rollback when problems arise.

Dead Code Elimination: Compile-Time Feature Gate

GrowthBook handles runtime feature flags. There's another gate layer at an earlier stage: compile time.

`process.env.USER_TYPE === 'ant'` is a compile-time constant. When building the external version, the bundler constant-folds it to `false`, then Dead Code Elimination (DCE) completely removes all `false` branch code.

The effect: **the external user's package physically contains no internal code whatsoever.** It's not "code is there but skipped" -- it genuinely doesn't exist. Smaller bundle size, faster startup, smaller attack surface. Even decompiling Claude Code's binary, external users can't see any trace of these internal features.

This leak was interesting precisely because what leaked was the pre-compilation source code, not the compiled artifact.

KAIROS: The Always-On AI Assistant

KAIROS is the most frequently referenced feature flag in the source code, appearing nearly 150 times. It represents Claude Code's ultimate vision: an AI assistant that **proactively observes, records, and acts without waiting for user input**.

Core features:

- **Append-only daily logs**: Records observations and decisions, forming the AI's own work journal
- **15-second blocking budget**: Bash commands running longer than 15 seconds in the main agent automatically move to the background, preventing user interaction blocking
- **Dedicated tools**: PushNotification (send user notifications), SendUserFile (send user files), SubscribePR (subscribe to PR updates)
- **Auto memory tidying** (autoDream): 24-hour time gate + 5-session minimum + tidying lock triple gate, preventing overly frequent automatic behavior
- **GitHub webhook integration**: Monitors repository events, responds automatically
- **Cron scheduled tasks**: Executes preset work on schedule

Launched via `claude assistant [sessionId]`. It has a completely different system prompt -- no longer "help users with programming" but "you are an autonomous agent, use tools to do useful work."

KAIROS's direction is clear: Claude Code isn't satisfied with a "you ask, it answers" tool positioning and is moving toward a **proactively thinking, continuously running AI companion**.

BUDDY: The Digital Pet System

This is probably the most unexpected discovery.

The `src/buddy/` directory hides a complete virtual pet system. Tamagotchi-style. Each user gets a companion sprite generated by a deterministic algorithm.

Technically interesting:

- **18 species**: duck, goose, blob, cat, dragon, octopus, owl, penguin, turtle, snail, ghost, axolotl, capybara, cactus, robot, rabbit, mushroom, chonk. Rarity from Common (60% weight) to Legendary (1% weight) across 5 tiers
- **6 eye styles**: `· ✦ × ◉ @ °`, down to each pet having different "eyes"
- **8 hats**: crown, tophat, propeller, halo, wizard, beanie, tinyduck (a tiny duck sitting on the head). Only uncommon and above get hats; commons are always `none`
- **1% shiny variant chance**
- **Deterministic randomization**: Uses Mulberry32 PRNG algorithm, seed = hash of user ID + salt `'friend-2026-401'`. Each user always gets the same pet, but different users get different ones
- **5 procedural attributes**: DEBUGGING, PATIENCE, CHAOS, WISDOM, SNARK -- each drawn from the same PRNG chain, with one "peak attribute" and one "dump stat," creating personality differences between pets
- **First hatching AI-generates personality**: Pets have "Bones" (deterministic part) and "Soul" (LLM-generated part). Bones are recalculated from the hash each time; the Soul is generated only on first hatching and permanently stored

An anti-cheat design worth noting: Bones data (including rarity and species) is never persistently stored -- it's recalculated from the user ID each time. Only Soul data (name and personality description) is stored in the config. Users can't edit their config file to turn a Common duck into a Legendary dragon. The code comment says it plainly: "users can't edit their way to a legendary."

BUDDY makes one thing clear: **even the most serious technology companies know products need a touch of fun.** An AI tool you want to open every day needs more than just powerful features -- it also needs a bit of personality.

ULTRAPLAN: Remote Planning Sessions

Offloads complex planning tasks to a remote Cloud Container Runtime (CCR), running Opus 4.6.

Core parameters:

- Up to **30 minutes of thinking time**
- Browser-based approval UI, real-time progress monitoring
- Polls for results every 3 seconds
- Special sentinel `__ULTRAPLAN_TELEPORT_LOCAL__` transfers results back to the terminal

This solves a real problem: some planning tasks are too complex and need the model to think for extended periods. Waiting 30 minutes in a local terminal is a terrible experience, but running in the background, monitoring progress via browser, and automatically sending results back to the terminal when done is a completely different experience.

Other Notable Flags

Flag	Feature	Significance
`tengu_penguins_off`	Penguin Mode (fast execution mode)	Dynamic switching between speed and quality
`tengu_amber_flint`	Swarm/Team capabilities	Multi-agent collaboration toggle
`tengu_scratch`	Shared scratch directory	File sharing between agents
`tengu_hive_evidence`	Verification Agent	Adversarial verification after completion

Feature Flags as Product Strategy

The 44 flags themselves reveal Claude Code's product evolution approach:

Progressive rollout. Features are first enabled internally (ant users), then gradually opened to external users after validation. Problems can be shut off in seconds. This lets the team experiment boldly, because the cost of failure is flipping a switch, not rolling back code.

A/B test driven. Extensive source code annotations mark "un-gate once validated on external via A/B" -- only fully opened after A/B validation on external users. Every feature release is a data-driven decision.

Model version binding. Many flags are tied to specific model versions (like "Capybara v8 thoroughness counterweight"). When models upgrade, related flags may need re-evaluation.

> KEY TAKEAWAY
>
> **Feature flags are essential infrastructure for AI products.** AI product behavior heavily depends on model versions, prompt strategies, and user feedback. Feature flags let you adjust product behavior without deploying code -- critical for fast-iterating AI products. The GrowthBook + DCE combination gives Claude Code's team two layers of control: runtime gradual rollout switches and compile-time hard isolation. The former is flexible; the latter is secure.

11 Two Claude Codes: Internal vs External

Internal vs External — Two Products in One Codebase

The source code is full of `process.env.USER_TYPE === 'ant'` checks. The Claude Code that Anthropic employees use differs significantly from your version. These differences reveal Anthropic's vision of how AI should ideally work.

Compile-Time Divergence: One Codebase, Two Products

`USER_TYPE` is a build-time constant (build-time `--define`). When building the internal version, it's `'ant'`; when building the external version, it's a different value. The key: **it's not a runtime environment variable but a compile-time constant.**

This means the bundler performs constant folding during packaging. All `process.env.USER_TYPE === 'ant'` in external builds are replaced with `false`, then Dead Code Elimination removes the entire `false` branch along with its required modules.

This gives Claude Code's team a unique ability: **maintaining two products in one codebase, with the external version physically containing no internal code.** It's not "code is there but if-skipped" -- it genuinely doesn't exist. External users can't see this code even by decompiling the binary.

Key Differences

Dimension	External Version	Internal Version
Code comments	No special requirements	Default to no comments; only add when the WHY is non-obvious
Honesty	No special requirements	Don't sugarcoat test results; don't imply unrun verifications passed
Output style	Concise and direct (4 lines)	Like writing to a person (20+ lines of detailed guidance)
Completion verification	No special requirements	Must actually verify before reporting completion
Output length	No hard limits	Between tool calls: max 25 words; final reply: max 100 words
Bug feedback	Points to GitHub Issues	Points to /issue and /share commands, can send to internal Slack
Verification Agent	Not enabled	A/B testing, automatic adversarial verification after completion

Quantitative Output Constraints

The "output length" row in the difference table deserves expansion. The internal version's prompt contains this:

```
// Numeric length anchors -- research shows ~1.2% output token
// reduction vs qualitative "be concise". Ant-only to measure
// quality impact first.
...(process.env.USER_TYPE === 'ant'
  ? ['Length limits: keep text between tool calls to ≤25 words.
      Keep final responses to ≤100 words unless the task
      requires more detail.']
  : [])
```

Note the comment: "research shows ~1.2% output token reduction vs qualitative 'be concise'."

This reveals Anthropic ran controlled experiments. The control group used qualitative instructions ("be concise," "keep it short"); the experimental group used quantitative instructions (max 25 words, max

100 words). **The result: quantitative constraints reduced output tokens by about 1.2% more than qualitative ones.**

1.2% sounds small, but at Anthropic's scale it's different. Claude Code processes an enormous volume of API calls daily -- 1.2% token reduction means significant cost savings and faster response times. And this optimization is zero-cost: no model changes, no code changes, just replacing "be concise" with "max 25 words."

The external version uses qualitative instructions: "Your responses should be short and concise." The internal version uses quantitative instructions plus a completely different output philosophy:

```
When sending user-facing text, you're writing for a person,
not logging to a console. Assume users can't see most tool
calls or thinking - only your text output.

Write user-facing text in flowing prose while eschewing
fragments, excessive em dashes, symbols and notation...

What's most important is the reader understanding your output
without mental overhead or follow-ups, not how terse you are.
```

The external version says "be concise"; the internal version says "write like you're writing to a person." The external version pursues "say less"; the internal version pursues "say it clearly." Two fundamentally different product philosophies. Anthropic clearly believes the latter is better. Why not give it to external users too? Likely because this prompt is longer, consuming more tokens, and needs internal validation before broader rollout.

Undercover Mode

When `USER_TYPE === 'ant'` and working in a public repository, Undercover Mode activates automatically. This mode sparked considerable community controversy.

Repository classification logic lives in `commitAttribution.ts`. The code maintains an allowlist `INTERNAL_MODEL_REPOS` of confirmed private internal repositories. The activation logic defaults to ON -- only positively confirmed internal repos disable it. Source code comment: "There is NO force-OFF. This guards against model codename leaks."

When activated:

- All model names are stripped from the system prompt
- Internal model codenames (animal names: Capybara, Tengu, etc.) are prohibited from leaking
- Unreleased model versions (like opus-4-7) are prohibited
- Internal tools and Slack references are hidden
- No traces indicating the commit came from AI

- Commit messages should read "like a human developer wrote them" -- describing only what code changed, not mentioning AI

The controversy centers on the last two points. The community considers this explicitly instructing AI to "pretend to be a human contributor," sparking discussions about AI transparency in open source.

Anti-Distillation Defense

Another community-discussed discovery.

Anthropic injects **fake tool definitions** into API requests. The purpose: if competitors try to train their own models by intercepting API requests (distillation), this fake data would poison the training set.

The source code also reveals another anti-distillation mechanism: **redacted thinking**. Through the `redact-thinking-2026-02-12` beta header, the model's reasoning process is edited or hidden in API responses. Even if competitors intercept API responses, they can't access the model's chain of thought -- and the chain of thought is precisely what distillation needs most.

Sentiment Detection

The source code has a surprising module: detecting negative sentiment in user messages using **regular expressions**. Not LLMs -- regex.

The community's first reaction was mockery: why would an LLM company use regex for sentiment detection? But the answer is pragmatic:

- Regex is roughly **10,000x faster** than the fastest LLM call
- Near-zero cost
- Deterministic results, suitable for metric comparison
- No additional API calls needed

After detecting negative sentiment, it only logs the event -- it doesn't change AI behavior. This is an **observability metric** for the team to understand user experience, not a feature.

This circles back to Chapter 8's theme: **not every problem needs the most advanced technology.** Regex sentiment detection isn't very accurate, but for the goal of "roughly understanding user sentiment trends," it's sufficient.

Adversarial Verification

A feature being A/B tested in the internal version: after completing complex implementations, an independent agent automatically performs **adversarial verification**.

Key design points:

- The verification agent and execution agent are completely independent
- The execution agent can't grade its own work
- Verification results have three levels: PASS, FAIL, PARTIAL
- On FAIL, must fix and re-verify until PASS
- The verification agent's command output must be re-run and compared against the execution agent's actual results

This is essentially the **AI version of code review**. One entity writes code, another reviews it. Except both are AI.

What the Differences Reveal

> **KEY TAKEAWAY**
>
> **The internal version exposes Anthropic's product ideal.** "Default to no comments" means they believe good code doesn't need comments. "Must verify before reporting completion" means they know AI tends to cut corners. "Write for people, not console logs" means they believe AI output should have warmth. "Max 25 words" instead of "be concise" means they use data-driven, not intuition-driven, product optimization.

If you're designing your own AI product, these internal version rules make great reference standards:

Require AI honesty. Explicitly state in instructions: "say uncertain when uncertain," "if tests fail, say they failed." Don't give AI wiggle room to fudge results.

Require AI verification. Don't let AI say "should be fine" -- require it to actually run verification and show results.

Treat AI output as text for humans. Not logs, not debug info, but structured, focused text that busy people can quickly understand.

Replace qualitative with quantitative constraints. "Max 100 words" is more effective than "keep it concise." Specific numbers give the model a clear anchor, while vague adjectives leave too much room for interpretation.

Compile-time isolation is safer than runtime checks. If your product has internal and external versions, stripping internal code at build time is far more reliable than runtime if-checks. The former guarantees the external version physically contains no sensitive code; the latter always has bypass potential.

12 The Harness Engineering Playbook

The Harness Engineering Playbook

The previous 11 chapters dissected the design decisions of Claude Code's six core systems. This final chapter distills those scattered insights into reusable principles. Not dogma -- rules of thumb.

Ten Design Principles

Principle 1: Simple Components + Smart Brain > Complexity Everywhere

Claude Code uses grep instead of RAG for search, plain text files instead of databases for memory, and mailbox files instead of message queues for communication. Each individual component is very simple, but combined with the LLM's comprehension, the overall performance surpasses approaches where every component is complex.

When you have a sufficiently powerful LLM as the system's core, surrounding components should be as simple as possible. Concentrating complexity at a single point (the model) is easier to manage than distributing it across ten points (each component having its own complexity).

Principle 2: Security Is the Foundation of Trust, Not a Limitation on Functionality

The permission system doesn't limit what AI can do -- it builds user trust in AI. Because you know there are four layers of security review, a circuit breaker, and a protected file list, you dare to enable Auto mode and let it operate freely.

When designing AI products, security system investment should match core feature investment. How much authority users are willing to give AI depends on how much confidence they have in the safety boundaries.

Principle 3: Remember Preferences, Forget Facts

The memory system only stores user preferences and judgments, not code and facts. Preferences are stable; facts change. Remembering "user prefers TypeScript" is far more reliable than remembering "function X is on line 30 of file Y."

Generalizing: **persistence should store "things that change slowly."** Model capabilities (search, comprehension) can retrieve "things that change fast" in real time.

Principle 4: Compression Is Lossy -- Design What Gets Lost

Context compression isn't a casual summary but a carefully designed nine-section structured extraction. Which information must be preserved (every user message), which can be condensed (detailed tool return data), and the organizing format (9 fixed sections) -- all explicitly specified.

If your AI application needs to handle long conversations, don't let the model freestyle "summarize the previous conversation." Give it a structured compression template, explicitly specifying what to keep and what to discard.

Principle 5: Tools Are the Boundary of Capability

All Claude Code operations go through tools indirectly. The model can't directly manipulate the file system -- it must call Read, Write, Bash, and other tools. This indirection isn't performance overhead; it's an architectural advantage.

Because all operations pass through the tool layer, you can do permission checking, logging, security review, and usage tracking at this layer. Tools define what AI can do and what it can't. **The best way to control AI isn't writing longer prompts -- it's designing better tools.**

Principle 6: Multi-Agent Collaboration Requires Organizational Design

Claude Code's multi-agent isn't simply forking a few processes. It has Team/Leader/Teammate hierarchy, independent git worktree isolation, async mailbox communication, and permission bubbling.

Multi-agent systems' difficulty isn't in technical implementation but in organizational design. How to split tasks, flow information, resolve conflicts, merge results -- these are identical to managing real human teams. Technical implementation is the easy part.

Principle 7: Feature Flags Manage Product Evolution

44 feature flags control unreleased features like KAIROS, BUDDY, and ULTRAPLAN. Each can be independently toggled, gray-released by user segment, and rolled back in seconds when problems arise.

For fast-iterating AI products, feature flags are essential infrastructure. They let teams experiment boldly because the cost of failure is flipping a switch, not rolling back code.

Principle 8: Internal and External Should Be the Same Product

Anthropic uses Claude Code themselves, and the internal version is stricter than the external one. This isn't privilege -- it's a source of product confidence. If you don't use your own AI product, how do you know if it's good?

The internal version's strict requirements (don't sugarcoat test results, don't write unnecessary comments, must verify before claiming completion) represent Anthropic's vision of how AI should ideally work. These requirements can also serve as reference benchmarks when designing your own AI products.

Principle 9: Caching Strategy Directly Impacts Costs

The system prompt's static/dynamic boundary, Deferred Tools' lazy loading, cross-user prompt cache reuse. These look like technical details but directly affect token consumption and cost of every API call.

When designing AI products, consider prompt caching strategy from day one. Put immutable parts up front for cache reuse; put variable parts at the end for on-demand loading. This might be the product's biggest cost optimization lever.

Principle 10: The 60/40 Rule

Claude Code's excellence is 60% model capability, 40% harness engineering.

This ratio says two things. **The model is still most important** -- choosing the right underlying model matters more than anything. But **the harness's 40% determines the ceiling of product experience** -- the same model with different harnesses becomes completely different products.

Don't neglect harness engineering because "the model matters most," and don't use a weak model because "the harness can compensate." Both are indispensable.

A Decision Checklist

If you're designing an AI product or AI Agent, these questions are worth asking at each design juncture:

Design Juncture	Claude Code's Choice	Ask Yourself
Search/Retrieval	grep, not RAG	Can the LLM's comprehension compensate for simple retrieval's shortcomings?
Persistence	Plain text files	Using the simplest storage approach, where's the bottleneck?
Memory	Preferences only	Does this information change fast or slow? Don't store the fast; only store the slow
Security	Four-layer review + circuit breaker	How much trust does the user need before granting AI this permission?
Context Management	Nine-section structured compression	What information absolutely cannot be lost during compression?
Multi-Agent	Team + worktree isolation	Do agents need isolation or sharing? How to resolve conflicts?
Cost	Prompt cache + lazy loading	Which parts of the prompt are unchanged across requests?
Evolution	44 feature flags	After a new feature goes live, can you roll back in seconds if problems arise?

Case Study: Building an AI Coding Tool from Scratch

Suppose you start building an AI coding tool from scratch tomorrow. Not a Claude Code clone, but your own tool for your own scenario. What can you learn from Claude Code's source code?

Your first decision: what approach for code retrieval?

Most people's first instinct is RAG -- embed the codebase, store in a vector database, use semantic search to find relevant code. This was 2024's hottest approach, looking the most "AI-native."

But Claude Code chose grep.

As discussed in Chapter 8: grep's results are precise, explainable, and zero-maintenance. The model is smart enough to understand raw code snippets from grep without needing vector search to "pre-understand" for it. The essence of this decision: **put complexity on the model side (comprehension), not the tool side (retrieval)**. When the underlying model is strong enough, the tool layer should be as simple as possible.

Your second decision: should you build a permission system?

Many developers think permissions are a "do it later" feature -- get the tool running first, add permissions gradually. But Claude Code had four-layer permission review, protected path lists, and circuit breakers from day one. As analyzed in Chapter 5, this isn't over-engineering -- it's **trust engineering**. Without a permission system, users won't dare enable Auto mode; without Auto mode, the tool's core value is diminished. The permission system isn't a shackle limiting AI -- it's a prerequisite for unlocking AI's capabilities.

Your third decision: what should the memory system store?

Intuition says store as much as possible -- remember every function the user modified, every conversation's full context, all files' modification history. But Chapter 6 shows us that Claude Code's CLAUDE.md only stores preferences: what coding style the user likes, what the project's tech stack is, which directories have special rules. Facts and code aren't stored because they change too fast -- storing them is actually more dangerous than not storing them (the model would use stale memories to override the actual current state).

Three decisions, three counterintuitive choices. Each choice is backed by the same mental model: **simple components + powerful model > complex components + any model**. The 11 chapters of this book dissect how this mental model unfolds across different dimensions.

Harness Engineering vs Prompt Engineering

Over the past two years, the entire industry has been discussing Prompt Engineering. How to write better system prompts, how to do few-shot, Chain-of-Thought vs Tree-of-Thought. All important, but Claude Code's source code reveals a bigger picture.

Prompt Engineering is **a subset of Harness Engineering**.

What is a Harness? It's the entire runtime environment built around the LLM. It includes:

Layer	Book Chapter	Focus
Prompt	Ch. 3 System Prompt	Model's role, behavioral norms, output format
Tools	Ch. 4 Tool System	What operations the model can perform, tool I/O design
Permissions	Ch. 5 Security Model	Which operations need approval, tiering, circuit breaking
Memory	Ch. 6 Memory System	What to persist, what to forget, how to recall
Context	Ch. 7 Context Management	How to compress, what to keep, what to discard
Multi-Agent	Ch. 9 Multi-Agent Architecture	Org structure, communication, isolation, conflict resolution

Most people only focus on the first layer -- Prompt. But Claude Code's source code proves the other five layers equally determine the product experience. The same model, same system prompt, with different Tool designs becomes a completely different product. Add different Permission strategies, and user trust levels diverge enormously. Pair with different Context management approaches, and long-conversation experience becomes night and day.

An analogy: **Prompt Engineering is like writing a recipe; Harness Engineering is like running a restaurant.** The recipe determines whether a dish tastes good, but a restaurant's experience goes far beyond the recipe. Sourcing, kitchen workflow, service processes, hygiene management, cost control -- every link affects the final experience. A restaurant with only a good recipe can't stay open, and an AI product with only a good prompt is the same.

Open Questions

This book analyzes Claude Code's source code as of April 2026. But harness engineering itself is rapidly evolving, and several questions remain open.

Will model capability upgrades make the harness obsolete? Some argue that when models are powerful enough, all this surrounding engineering becomes unnecessary. I lean toward no. Many problems the harness solves are unrelated to model capability. Permission control is a product requirement, not a model deficiency. Context windows will always be finite, so compression strategies will always be needed. Multi-agent organizational design is a structural problem of human-AI collaboration that doesn't auto-resolve with smarter models. Stronger models will change how harnesses are implemented (e.g., simpler tool descriptions sufficing), but won't eliminate the harness's necessity.

The future of open-source harness frameworks? Currently every AI product builds its own wheels. Claude Code's harness, Cursor's harness, Windsurf's harness -- similar underlying ideas but completely different implementations. Will a "harness framework" emerge, like React for frontend or Rails for backend, providing standardized tool systems, permission systems, and memory systems? Perhaps, but the harness's uniqueness lies in its tight coupling to specific product forms. A coding tool's harness and a customer service bot's harness differ too much to cover with a single framework.

Does every AI application need its own harness? If you're only using the model for text generation, probably not. But if your AI product involves tool invocation, state management, user interaction, or long-term memory -- any one of these -- you're doing harness engineering, even if you haven't realized it yet. Claude Code's value lies in making this implicit engineering explicit, letting us see what a mature AI product does at every layer.

Final Thoughts

Harness engineering is still a very early discipline. No textbooks, no widely accepted best practices, no standardized frameworks. Claude Code's source code is the best reference implementation available to date, but it's also just one possible approach.

What this book attempts to do is: translate a multi-billion-dollar AI product's design blueprint into understandable, borrowable engineering principles. Not for you to copy Claude Code's implementation (that's pointless -- your tech stack and scenario are different), but to understand the Why behind each design decision, then apply those Whys to your own AI products.

If this book causes you to ask one extra question at your next design junction -- "why did Claude Code choose a different approach?" -- then it has served its purpose.

Follow Huashu

WeChat · Bilibili · YouTube · X

WeChat Official Account "Huashu": Frontline practice as an AI Native Coder

Bilibili "Huashu": In-depth AI tool reviews & tutorials

YouTube @AIchain: AI programming tutorials

X/Twitter @AIchainHust: AI insights & updates

Bilibili · X/Twitter · YouTube · Website

Claude Code: Source Code Analysis · Orange Book Series · April 2026
Based on Claude Code v2.1.88 source code analysis